



CS601T: Deep Learning

Programming Assignment II

Fully Connected Neural Networks Trained from Scratch with Stochastic Gradient Descent Backpropagation

Nimitt Jain (CS24BT048), Dattaraj Saudagar (CS24BT037), and Omkar Mulje (CS24BT059)

Group 12
Department of Computer Science and Engineering,
Indian Institute of Technology Dharwad, Karnataka, India

Classification: LS and NLS datasets | Regression: univariate and bivariate datasets

1 Implementation and Experimental Setup

1.1 Model and Notation

Every model in this report is a fully connected feedforward neural network (FCNN) written from scratch. No neural-network, gradient-descent or machine-learning library is used: `numpy` only performs array arithmetic and `matplotlib` only renders the figures. The notation below is the one used in the course slides.

- Training data $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$, with $\mathbf{x}_n \in \mathbb{R}^d$ and $\mathbf{y}_n \in \mathbb{R}^K$: d input nodes (index i), J hidden nodes (index j) and K output nodes (index k).
- $\mathbf{W}^{(h)}$ holds the weights $w_{ij}^{(h)}$ between the input and the hidden layer; $\mathbf{W}^{(o)}$ holds the weights $w_{jk}^{(o)}$ between the hidden and the output layer.
- a_{nj} is the net input of hidden node j and $h_{nj} = g(a_{nj})$ its output; $a_{nk}^{(o)}$ is the net input of output node k and $\hat{y}_{nk} = f(a_{nk}^{(o)})$ its output.
- The bias is the weight of index 0, exactly as in the slides' sums that begin at $j = 0$:

$$a_k^{(o)} = \sum_{j=0}^J w_{jk}^{(o)} h_j, \quad \text{with} \quad h_0 = 1. \quad (1)$$

The hidden nodes use the logistic activation, whose derivative is written through the node's own output:

$$g(a_j) = \frac{1}{1 + e^{-a_j}}, \quad \frac{\partial g(a_j)}{\partial a_j} = g(a_j)(1 - g(a_j)). \quad (2)$$

The output nodes use the logistic function for classification, where the targets are 1-of- K coded, and the identity for regression:

$$f(a_k^{(o)}) = \begin{cases} \frac{1}{1 + e^{-a_k^{(o)}}} & \text{(classification),} \\ a_k^{(o)} & \text{(regression).} \end{cases} \quad (3)$$

With two hidden layers the same hidden-layer rule is simply applied again, layer by layer.

1.2 Learning Algorithm: Backpropagation in Pattern Mode (SGD)

Training follows the parameter-learning procedure taught in class — backpropagation in pattern mode, i.e. stochastic gradient descent — with the squared error as the instantaneous loss function, as the assignment requires. The seven steps below are implemented one-for-one in `fcnn.py`.

1. Initialise $\mathbf{W}^{(h)}$ and $\mathbf{W}^{(o)}$ with random values.
2. Choose a training example $\mathbf{x}_n$ (the examples are presented in a fresh random order every epoch).
3. *Forward computation*: compute the output of all output neurons, $\hat{y}_{nk}$ for $k = 1, \dots, K$.
4. *Backward operation*: the formulas are given in Eqs. (6)–(11) below.
5. Repeat steps 2 to 4 until all the training examples have been presented once — one *epoch*. The weights are therefore updated after every single example, which is what makes this stochastic rather than batch gradient descent.
6. Compute the average error E_{av} . This is the quantity plotted against epochs throughout this report.
7. Repeat steps 2 to 6 until the stopping criterion is satisfied.

The instantaneous error of example n and the average error over an epoch are

$$E_n = \frac{1}{2} \sum_{k=1}^K (y_{nk} - \hat{y}_{nk})^2, \quad (4)$$

$$E_{av} = \frac{1}{N} \sum_{n=1}^N E_n. \quad (5)$$

The step-4 formulas, with $0 < \eta \leq 1$ the learning rate, are as follows. For the *hidden* $\rightarrow$ *output* weights,

$$\delta_{nk}^{(o)} = (y_{nk} - \hat{y}_{nk}) \frac{\partial f(a_{nk}^{(o)})}{\partial a_{nk}^{(o)}}, \quad (6)$$

$$\Delta w_{jk}^{(o)} = \eta \delta_{nk}^{(o)} h_{nj}, \quad (7)$$

$$w_{jk}^{(o)} \leftarrow w_{jk}^{(o)} + \Delta w_{jk}^{(o)}, \quad (8)$$

and for the *input* $\rightarrow$ *hidden* weights,

$$\delta_{nj}^{(h)} = \left[\sum_{k=1}^K \delta_{nk}^{(o)} w_{jk}^{(o)} \right] \frac{\partial g(a_{nj})}{\partial a_{nj}}, \quad (9)$$

$$\Delta w_{ij}^{(h)} = \eta \delta_{nj}^{(h)} x_i, \quad (10)$$

$$w_{ij}^{(h)} \leftarrow w_{ij}^{(h)} + \Delta w_{ij}^{(h)}. \quad (11)$$

For the two-hidden-layer networks the step-4 hidden-layer rule is applied again, layer by layer, from the last hidden layer back to the first: the sum over k is then taken over the nodes of the layer immediately above.

1.3 Initialisation, Standardisation and Convergence

Initialisation. As prescribed in class, the weight values are drawn randomly and scaled by $1/\sqrt{n}$, where n is the number of nodes feeding the weight (for $\mathbf{W}^{(o)}$ that is J , the number of hidden nodes). This keeps the initial net inputs a_j , a_k inside the active (non-saturated) region of the logistic function. Four schemes are implemented,

$$w \sim \frac{\mathcal{N}(0, 1)}{\sqrt{n}}, \quad w \sim \mathcal{U}\left(-\frac{1}{\sqrt{n}}, +\frac{1}{\sqrt{n}}\right), \quad (12)$$

$$w \sim \mathcal{U}\left(-\frac{2}{\sqrt{n}}, +\frac{2}{\sqrt{n}}\right), \quad w \sim \mathcal{U}\left(\pm \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right), \quad (13)$$

and the third of them — a uniform distribution two standard deviations wide — is used throughout, because it gave the fastest descent of E_{av} when the four were measured against each other on all four datasets over three seeds (`compare_initialisation.py`).

Normalisation of the input. As prescribed in class, the inputs are z-score normalised — and, for regression, so are the targets — using *training-set statistics only*:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \quad \mu_i, \sigma_i \text{ estimated on the training split alone.} \quad (14)$$

This matters in practice: the logistic units saturate on the raw LS inputs ($x_1 \approx 20$) and on the raw bivariate targets (up to ≈ 90), and SGD then barely moves. Every RMSE and %RMSE reported below is computed after inverse-transforming the predictions back to the original units, so the values remain directly comparable with the Assignment-1 numbers.

Stopping criterion. As taught in class, training stops when a fixed number of epochs is reached, or when

$$|E_{\text{av}}(t) - E_{\text{av}}(t-1)| < \varepsilon, \quad \varepsilon = 10^{-8}, \quad (15)$$

holds for several consecutive epochs. Because the weights are updated after every single example the E_{av} curve is noisy, so a single flat epoch — or the larger $\varepsilon = 10^{-3}$ given as an example on the slide — halts training on noise long before it has converged. In addition, training stops if the validation error has not improved for that many epochs, and the weights of the epoch with the lowest validation error are restored (early stopping); this last part uses the validation set the assignment asks for and is not part of the slide’s criterion.

Model selection. For classification the architecture with the highest validation accuracy is selected, ties broken by the lower validation E_{av} ; for regression the architecture with the lowest validation RMSE is selected. All the plots requested “for the best architecture” use that model, and the test set is touched only once, after selection.

1.4 Test Phase

For a test example $\mathbf{x}$ the output of each neuron in the output layer is computed with the weights obtained by training. For **classification** the class label assigned to $\mathbf{x}$ is the index of the largest output,

$$\hat{y}_k = f(a_k^{(o)}) = f\left(\sum_{j=0}^J w_{jk}^{(o)} h_j\right), \quad \text{class}(\mathbf{x}) = \arg \max_{k=1, \dots, K} \hat{y}_k, \quad (16)$$

with f the logistic function. For **regression** the output node is linear, so

$$\hat{y}_k = a_k^{(o)} = \sum_{j=0}^J w_{jk}^{(o)} h_j, \quad (17)$$

and this value is inverse-transformed back to the original target units before the RMSE is computed. The two error measures used for regression are

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2}, \quad \% \text{RMSE} = 100 \times \frac{\text{RMSE}}{y_{\max} - y_{\min}}. \quad (18)$$

1.5 Data

The datasets are the ones issued to Group 12 for Assignment-1. Each dataset is split once, with a fixed seed, into 60 % training / 20 % validation / 20 % test data; for the classification datasets the split is done independently within each class so that the proportion holds per class. The split is cached, so changing the architecture never reshuffles the data.

Table 1. The fixed 60/20/20 split used by every experiment.

Dataset	Training	Validation	Test	Total
LS classification (3 classes, 2-D)	900	300	300	1500
NLS classification (3 classes, 2-D)	900	300	300	1500
Univariate regression (1-D input)	601	200	200	1001
Bivariate regression (2-D input)	6121	2040	2040	10201

2 Classification — Dataset 1 (LS, Linearly Separable)

Model: an FCNN with one hidden layer, logistic hidden and output nodes, three output nodes with 1-of- K coded targets, trained by SGD backpropagation with the squared-error loss and learning rate $\eta = 0.05$. 4 architectures were trained and compared on the validation set.

2.1 Results for the Different Architectures (Validation Data)

Table 2. Validation-set performance of every architecture that was tried. The selected architecture is highlighted.

Hidden nodes	Architecture	Params	Epochs	Train acc.%	Val acc.%	Mean prec.	Mean recall	Mean F
8	2-8-3	51	150	100.00	100.00	1.0000	1.0000	1.0000
16	2-16-3	99	150	100.00	100.00	1.0000	1.0000	1.0000
32	2-32-3	195	150	100.00	100.00	1.0000	1.0000	1.0000
64	2-64-3	387	150	100.00	100.00	1.0000	1.0000	1.0000

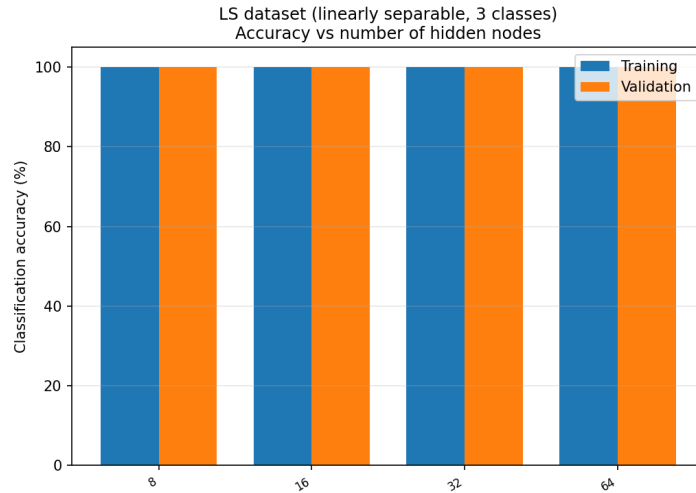


Fig. 1. Training and validation accuracy against the number of hidden nodes.

Average error against epochs for each architecture that was tried, one graph per architecture. They are deliberately not overlaid on a single axis: with several architectures the curves sit on top of one another and none of them can be read.

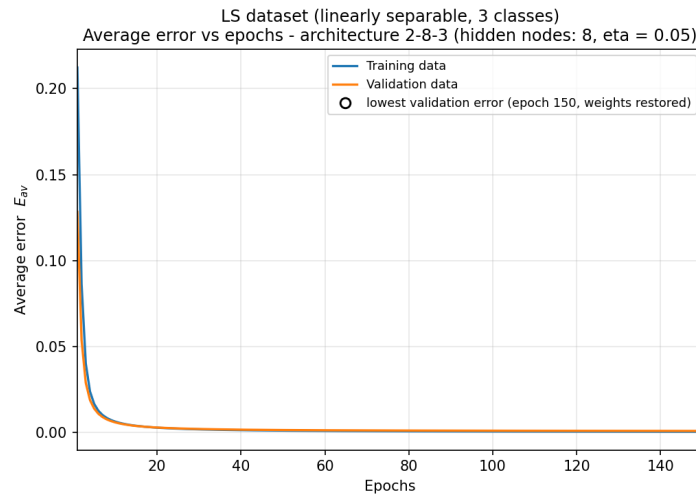


Fig. 2. Average error against epochs, architecture 2-8-3 (hidden nodes: 8).

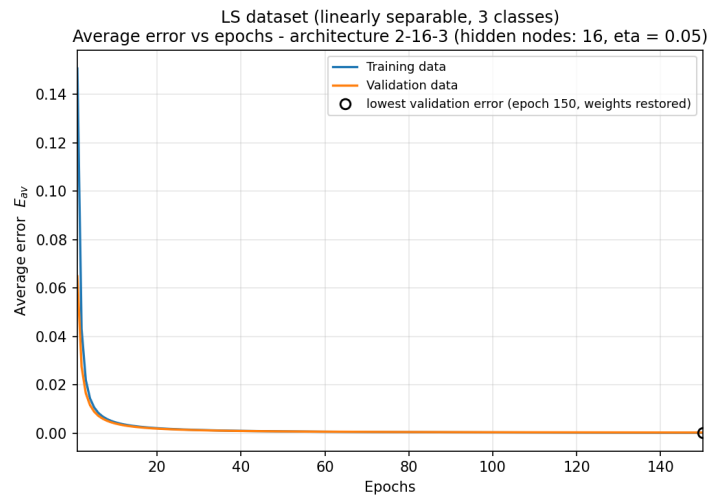


Fig. 3. Average error against epochs, architecture 2-16-3 (hidden nodes: 16).

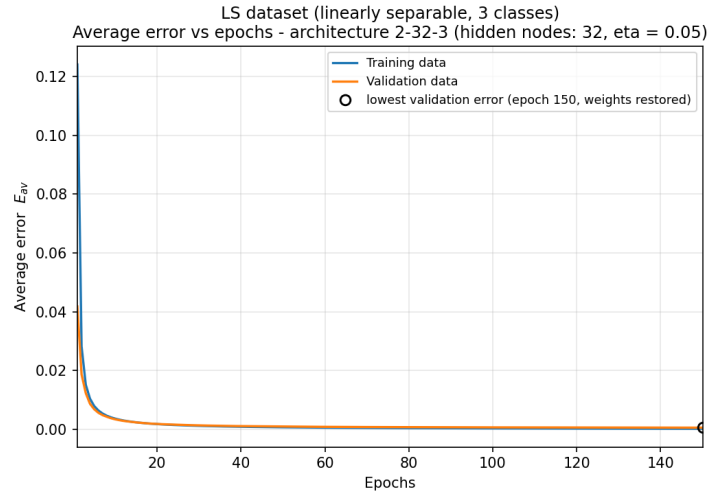


Fig. 4. Average error against epochs, architecture 2-32-3 (hidden nodes: 32).

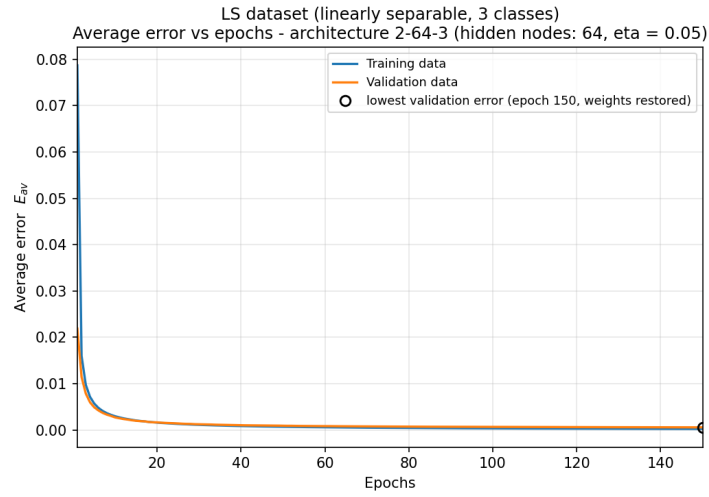


Fig. 5. Average error against epochs, architecture 2-64-3 (hidden nodes: 64).

2.2 Confusion Matrix and Per-Class Metrics on Validation Data, for Every Architecture

Architecture 2-8-3 (hidden nodes 8) — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Architecture 2-16-3 (hidden nodes 16) (selected) — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Architecture 2-32-3 (hidden nodes 32) — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Architecture 2-64-3 (hidden nodes 64) — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

2.3 Best Architecture: 2-16-3

The architecture selected on the validation set is **2-16-3** (hidden nodes: 16), trained for 150 epochs. All the plots below are for this model.

Average error against epochs.

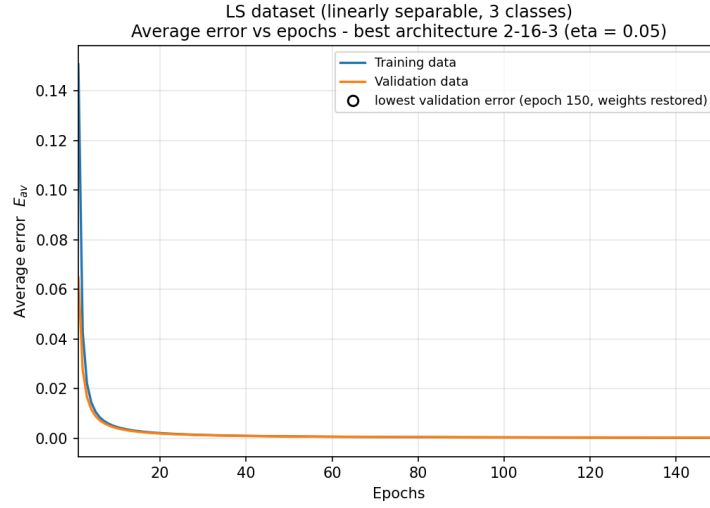


Fig. 6. Average error E_{av} against epochs for the best architecture.

Decision region superimposed on the training data.

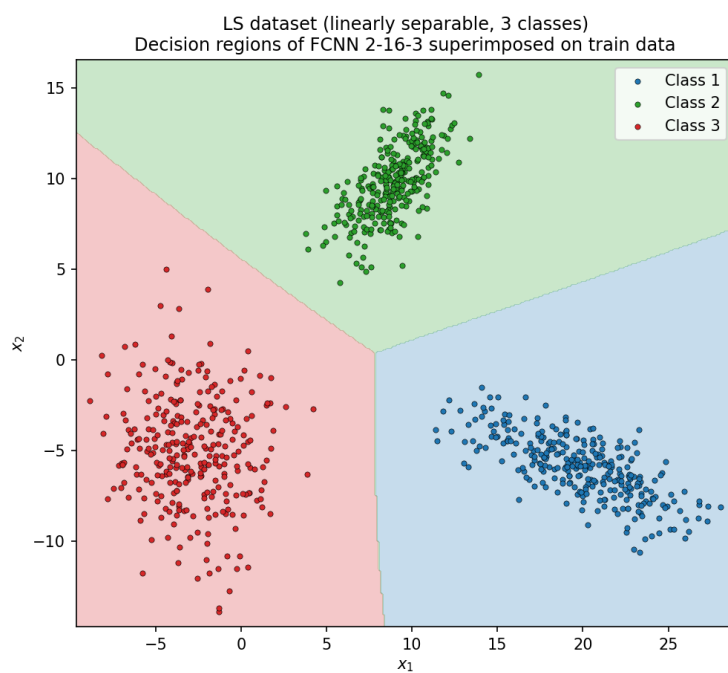


Fig. 7. Decision regions of the best FCNN, superimposed on the training data.

Confusion matrix and metrics for the best architecture. **Training data** — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	300	0	0
True 2	0	300	0
True 3	0	0	300

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Validation data — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Test data — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

2.4 Outputs of the Hidden Nodes and Output Nodes

Every hidden node and every output node is plotted in its own figure: the two axes in the base plane are the input variables x_1 and x_2 of each example and the vertical axis is the output of that node. The selected network has 19 nodes in total, so the complete set is 57 figures (one per node per split); they are all written to `outputs/classification_ls/best_architecture/node_outputs/`. The first 6 nodes of each layer are reproduced below for each split.

Hidden layer 1 — train data.

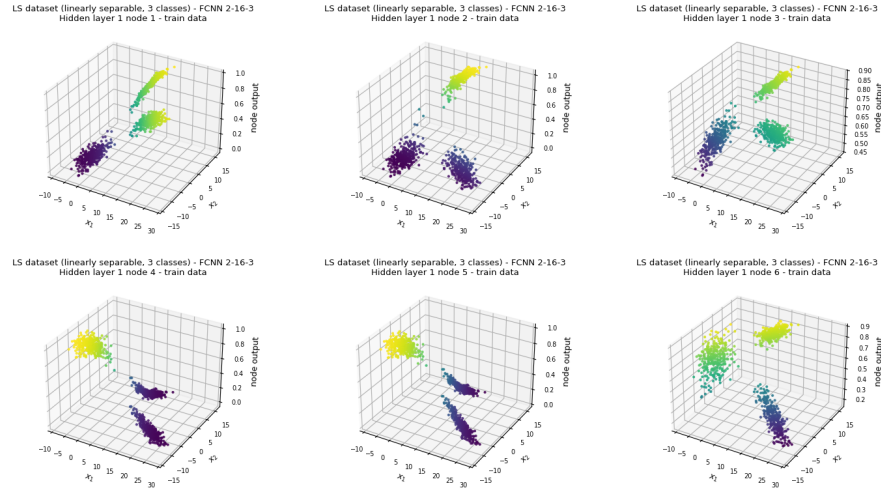


Fig. 8. Outputs of hidden layer 1 nodes 1–6 of 16 (train data). Each node has its own figure; all 16 are in `outputs/classification_ls/best_architecture/node_outputs/hidden_layer1/train/`.

Hidden layer 1 — val data.

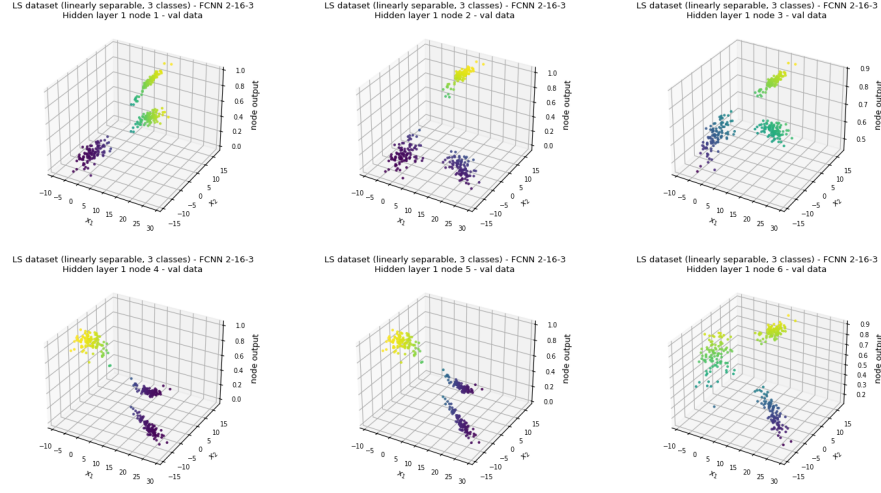


Fig. 9. Outputs of hidden layer 1 nodes 1–6 of 16 (val data). Each node has its own figure; all 16 are in `outputs/classification_ls/best_architecture/node_outputs/hidden_layer1/val/`.

Hidden layer 1 — test data.

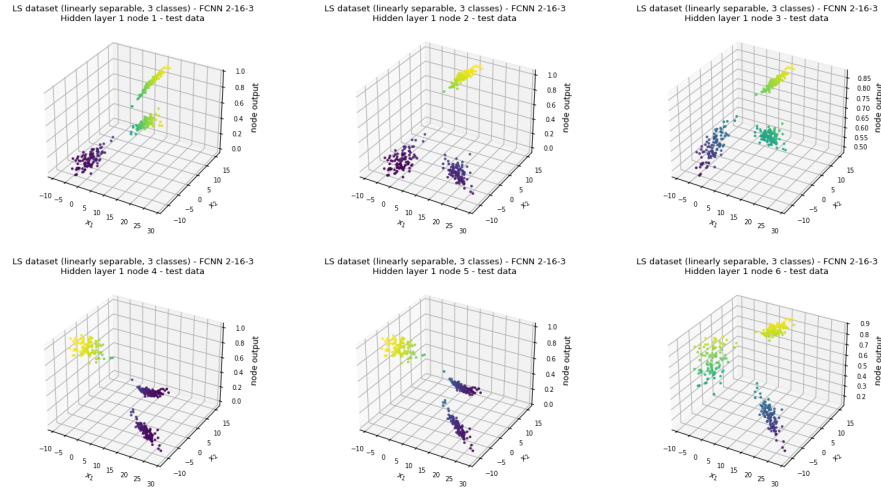


Fig. 10. Outputs of hidden layer 1 nodes 1–6 of 16 (test data). Each node has its own figure; all 16 are in `outputs/classification_ls/best_architecture/node_outputs/hidden_layer1/test/`.

Output layer — train data.

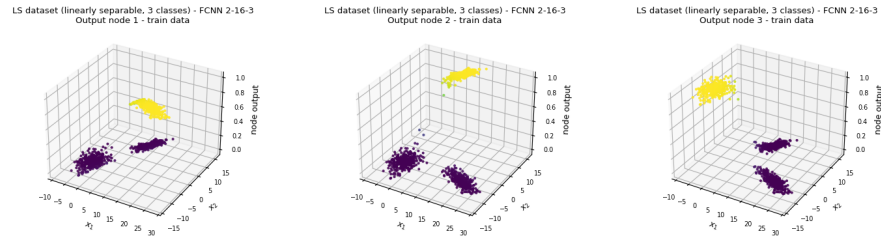


Fig. 11. Outputs of all 3 output nodes (train data), one figure per node.

Output layer — val data.

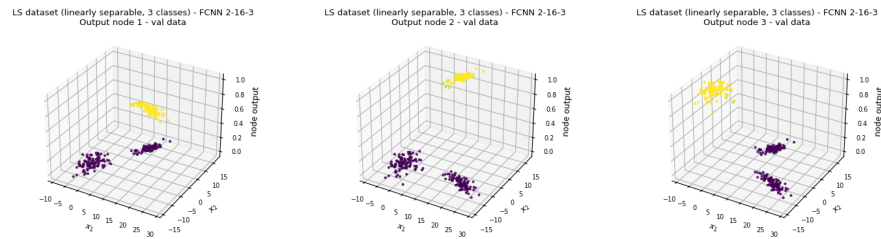


Fig. 12. Outputs of all 3 output nodes (val data), one figure per node.

Output layer — test data.

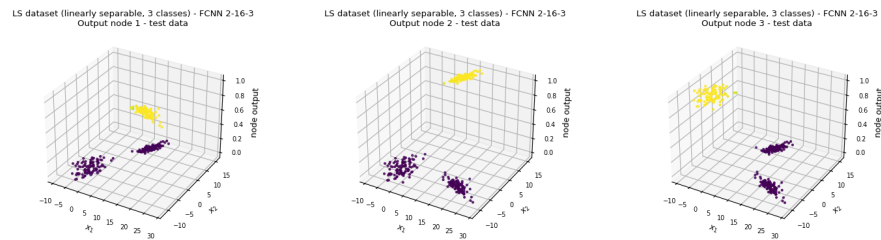


Fig. 13. Outputs of all 3 output nodes (test data), one figure per node.

2.5 Comparison with the Single-Neuron Model (Assignment-1)

The Assignment-1 model — one perceptron per pair of classes, combined by one-against-one majority voting — was retrained on exactly the same 60/20/20 split and scored with the same from-scratch metric code, so the two rows below are directly comparable.

Table 3. FCNN against the Assignment-1 single-neuron model.

Model	Val acc.%	Test acc.%	Test mean prec.	Test mean recall	Test mean F
Single neuron (one-against-one perceptron)	100.00	100.00	1.0000	1.0000	1.0000
FCNN 2-16-3 (Assignment-2)	100.00	100.00	1.0000	1.0000	1.0000

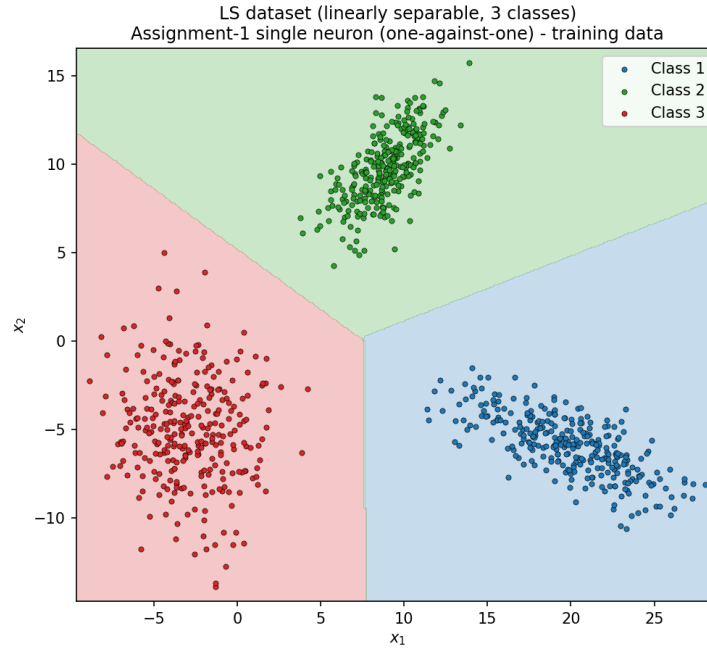


Fig. 14. Decision regions of the Assignment-1 single-neuron model on the same training data.

2.6 Inferences

- The three classes of the LS dataset form compact, well-separated clusters, so every architecture that was tried reached 100.00% validation accuracy, and the accuracies tie.

- In terms of classification accuracy alone the small network 2-8-3 (51 weights) is already sufficient, and by Occam's razor it is the model one would deploy: beyond it, extra hidden nodes add parameters without adding accuracy. The architecture reported as "best" (2-16-3) is the one the selection rule returns because, among those tied on accuracy, it has the smallest validation average error - its output activations sit closest to the 1-of-K targets.
- The single neuron of Assignment-1 also achieves 100.00% on this test data (+0.00 percentage points difference against the FCNN). This is the important negative result of this dataset: when the classes are linearly separable, the extra representational power of hidden layers buys nothing, and the simpler model should be preferred.
- The decision-region plot shows almost straight boundaries meeting near the centre of the input space. Although the FCNN *can* bend its boundaries, training drives it to an essentially piecewise-linear solution, because that is already enough to drive the squared error to a very small value.
- The hidden-node output plots show sigmoid ridges oriented in different directions: each hidden node is effectively a soft linear discriminant, and the output layer recombines a small number of them. Several hidden nodes in the larger networks produce almost the same surface, i.e. they are redundant.
- The average-error curve of the best architecture falls steeply during the first epochs and then flattens; training ran for 150 epochs before the convergence criterion stopped it. Training and validation curves stay close together, so there is no sign of overfitting at this network size.
- Training, validation and test metrics of the selected model agree closely, which is what one expects when the model is selected on data that is disjoint from the test set and the three splits are drawn from the same distribution.

3 Classification — Dataset 2 (NLS, Nonlinearly Separable)

Model: an FCNN with two hidden layers, logistic hidden and output nodes, three output nodes with 1-of- K coded targets, trained by SGD backpropagation with the squared-error loss and learning rate $\eta = 0.05$. 4 architectures were trained and compared on the validation set.

3.1 Results for the Different Architectures (Validation Data)

Table 4. Validation-set performance of every architecture that was tried. The selected architecture is highlighted.

Hidden nodes	Architecture	Params	Epochs	Train acc.%	Val acc.%	Mean prec.	Mean recall	Mean F
16-8	2-16-8-3	211	300	100.00	100.00	1.0000	1.0000	1.0000
32-16	2-32-16-3	675	300	100.00	100.00	1.0000	1.0000	1.0000
64-16	2-64-16-3	1283	300	100.00	100.00	1.0000	1.0000	1.0000
64-32	2-64-32-3	2371	300	100.00	100.00	1.0000	1.0000	1.0000

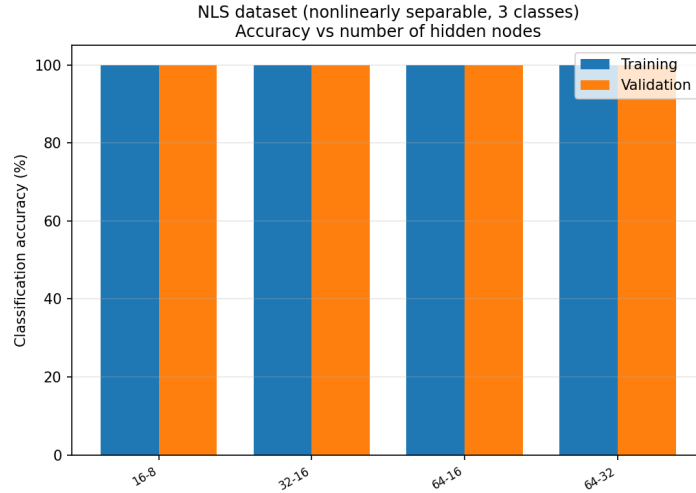


Fig. 15. Training and validation accuracy against the number of hidden nodes.

Average error against epochs for each architecture that was tried, one graph per architecture. They are deliberately not overlaid on a single axis: with several architectures the curves sit on top of one another and none of them can be read.

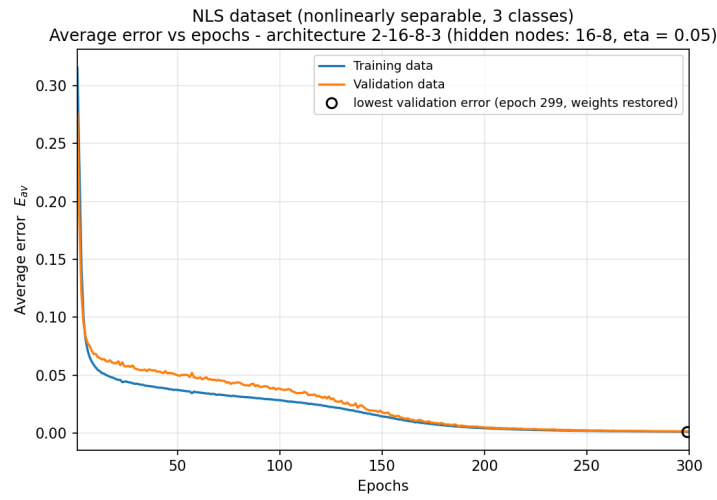


Fig. 16. Average error against epochs, architecture 2-16-8-3 (hidden nodes: 16-8).

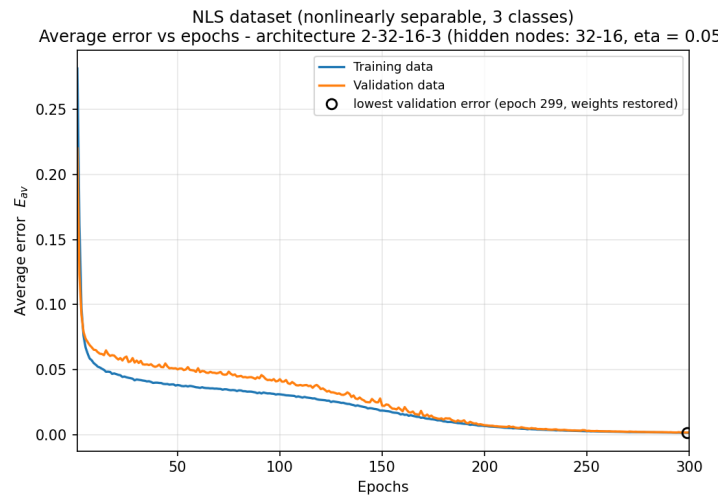


Fig. 17. Average error against epochs, architecture 2-32-16-3 (hidden nodes: 32-16).

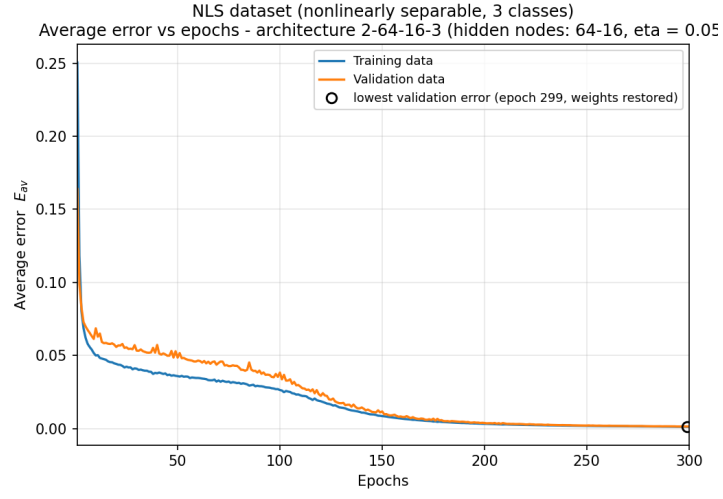


Fig. 18. Average error against epochs, architecture 2-64-16-3 (hidden nodes: 64-16).

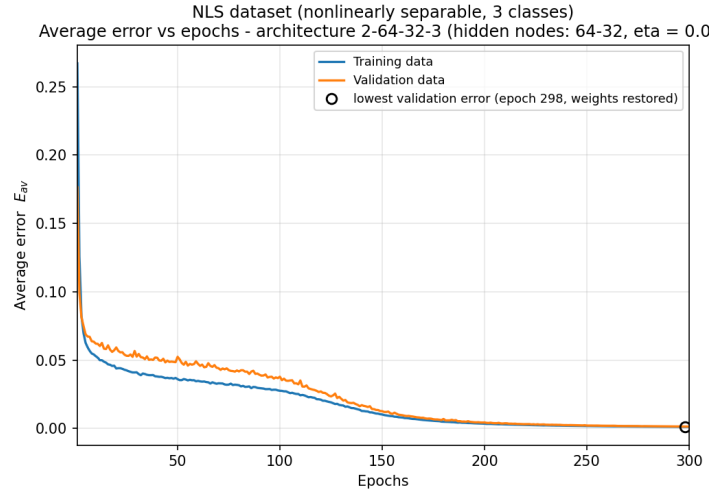


Fig. 19. Average error against epochs, architecture 2-64-32-3 (hidden nodes: 64-32).

3.2 Confusion Matrix and Per-Class Metrics on Validation Data, for Every Architecture

Architecture 2-16-8-3 (hidden nodes 16-8) — classification accuracy = 100.00%

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Architecture 2-32-16-3 (hidden nodes 32-16) — classification accuracy = 100.00%

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Architecture 2-64-16-3 (hidden nodes 64-16) (selected) — classification accuracy = 100.00%

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Architecture 2-64-32-3 (hidden nodes 64-32) — classification accuracy = 100.00%

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

3.3 Best Architecture: 2-64-16-3

The architecture selected on the validation set is **2-64-16-3** (hidden nodes: 64-16), trained for 300 epochs. All the plots below are for this model.

Average error against epochs.

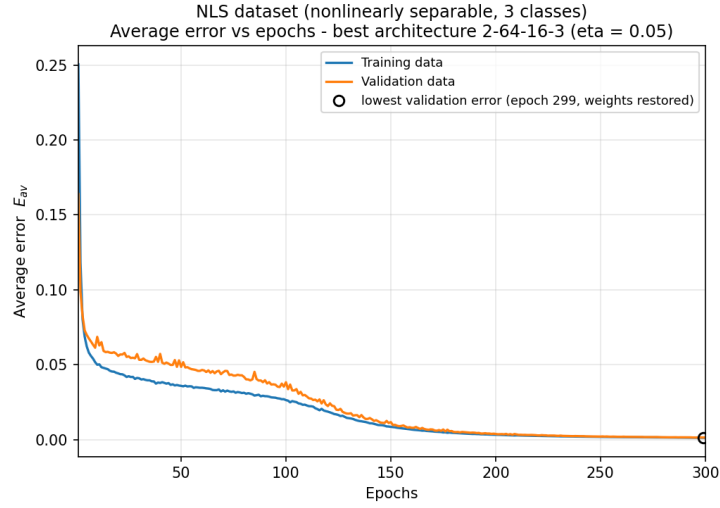


Fig. 20. Average error E_{av} against epochs for the best architecture.

Decision region superimposed on the training data.

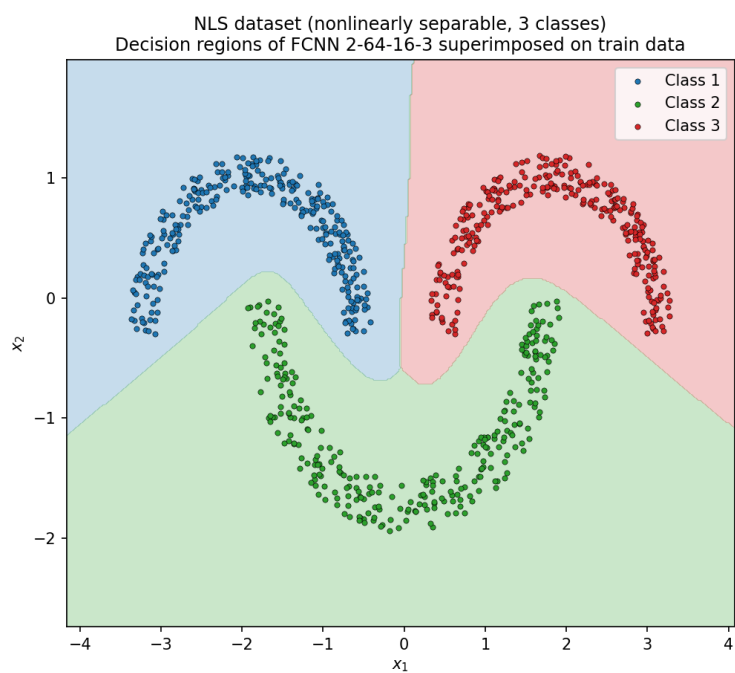


Fig. 21. Decision regions of the best FCNN, superimposed on the training data.

Confusion matrix and metrics for the best architecture. **Training data** — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	300	0	0
True 2	0	300	0
True 3	0	0	300

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Validation data — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

Test data — classification accuracy = **100.00%**

	Pred 1	Pred 2	Pred 3
True 1	100	0	0
True 2	0	100	0
True 3	0	0	100

Class	Precision	Recall	F-measure
1	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

3.4 Outputs of the Hidden Nodes and Output Nodes

Every hidden node and every output node is plotted in its own figure: the two axes in the base plane are the input variables x_1 and x_2 of each example and the vertical axis is the output of that node. The selected network has 83 nodes in total, so the complete set is 249 figures (one per node per split); they are all written to `outputs/classification_nls/best_architecture/node_outputs/`. The first 6 nodes of each layer are reproduced below for each split.

Hidden layer 1 — train data.

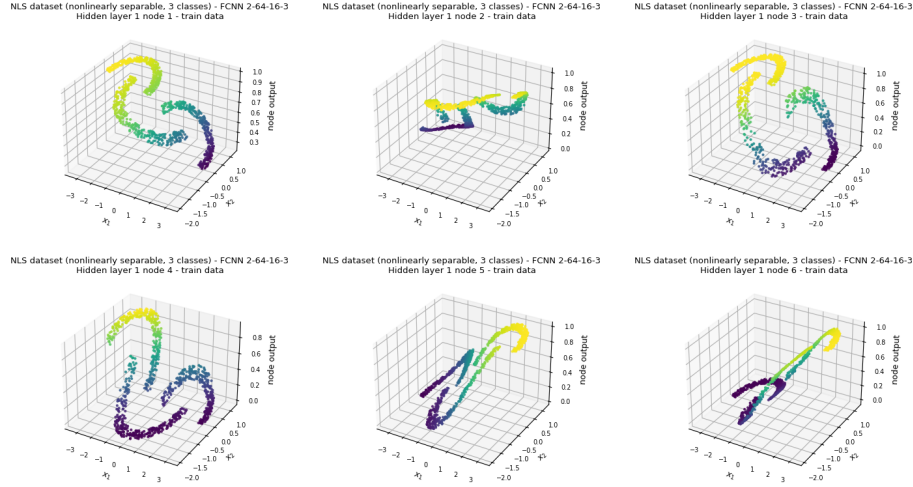


Fig. 22. Outputs of hidden layer 1 nodes 1–6 of 64 (train data). Each node has its own figure; all 64 are in `outputs/classification_nls/best_architecture/node_outputs/hidden_layer1/train/`.

Hidden layer 1 — val data.

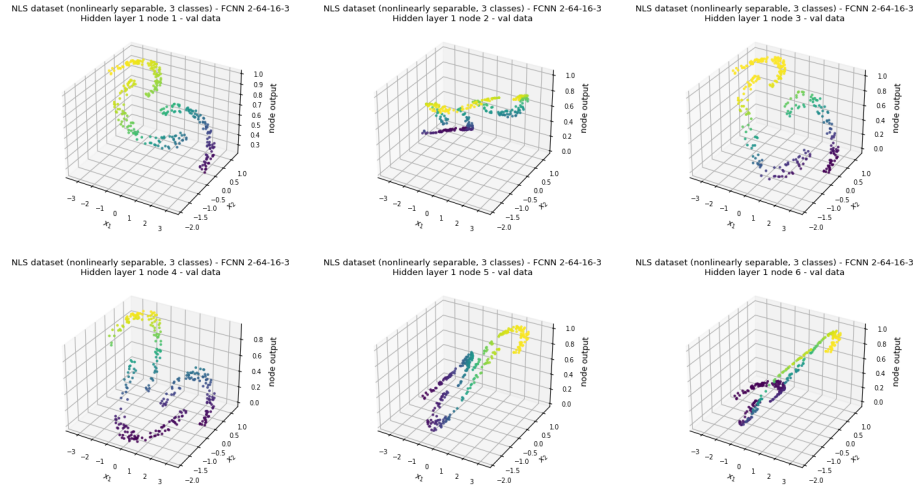


Fig. 23. Outputs of hidden layer 1 nodes 1–6 of 64 (val data). Each node has its own figure; all 64 are in `outputs/classification_nls/best_architecture/node_outputs/hidden_layer1/val/`.

Hidden layer 1 — test data.

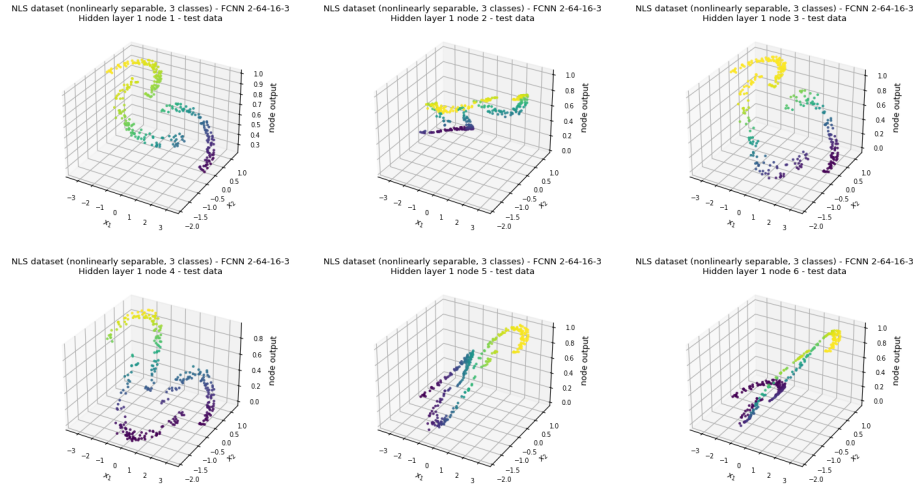


Fig. 24. Outputs of hidden layer 1 nodes 1–6 of 64 (test data). Each node has its own figure; all 64 are in `outputs/classification_nls/best_architecture/node_outputs/hidden_layer1/test/`.

Hidden layer 2 — train data.

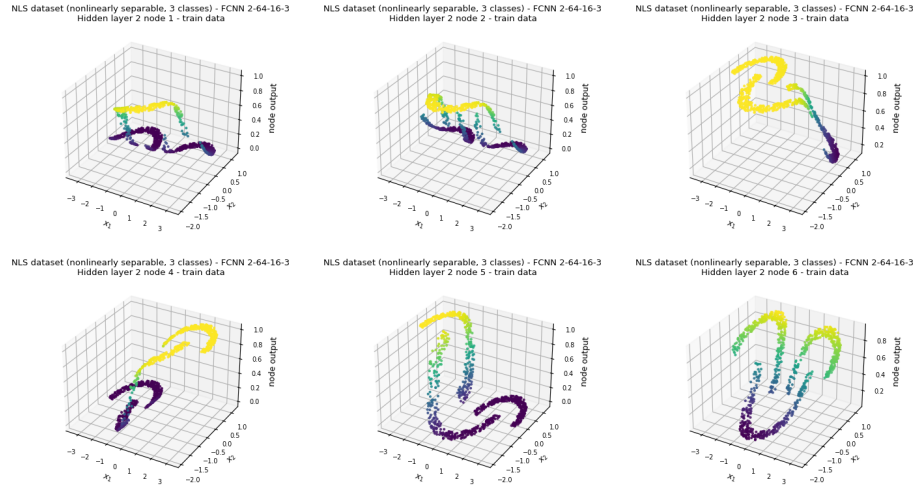


Fig. 25. Outputs of hidden layer 2 nodes 1–6 of 16 (train data). Each node has its own figure; all 16 are in `outputs/classification_nls/best_architecture/node_outputs/hidden_layer2/train/`.

Hidden layer 2 — val data.

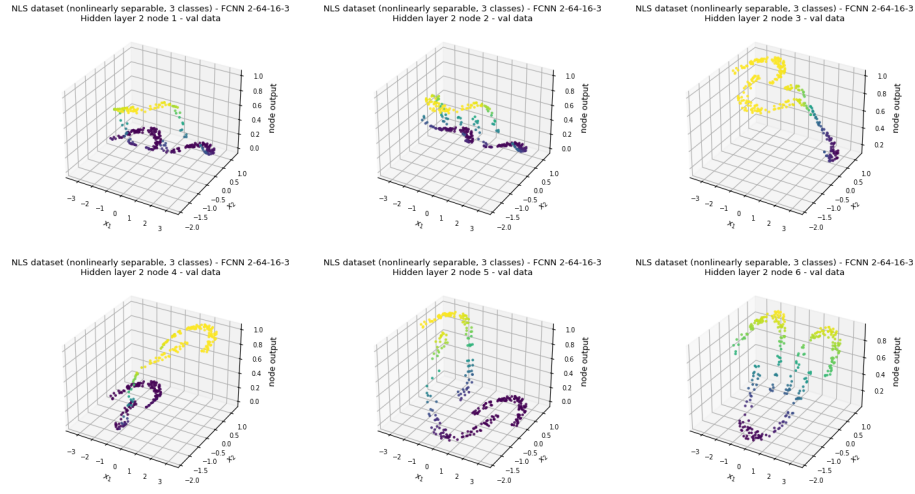


Fig. 26. Outputs of hidden layer 2 nodes 1–6 of 16 (val data). Each node has its own figure; all 16 are in `outputs/classification_nls/best_architecture/node_outputs/hidden_layer2/val/`.

Hidden layer 2 — test data.

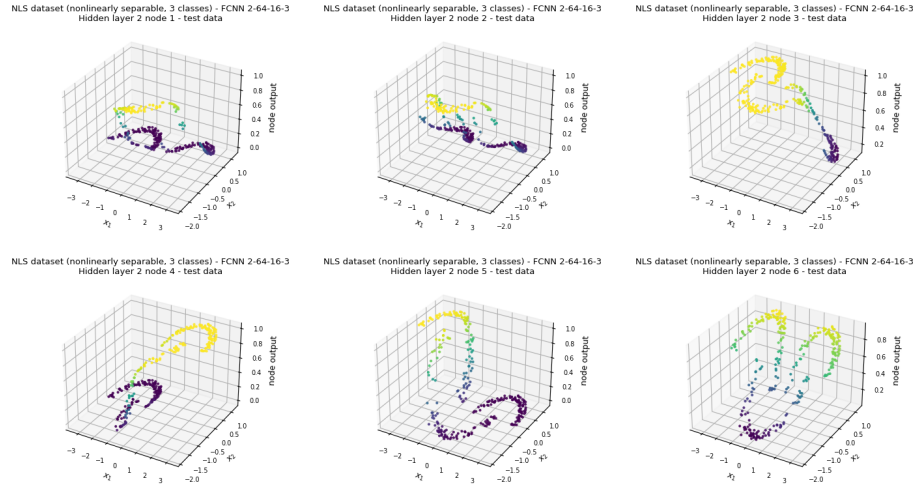


Fig. 27. Outputs of hidden layer 2 nodes 1–6 of 16 (test data). Each node has its own figure; all 16 are in `outputs/classification_nls/best_architecture/node_outputs/hidden_layer2/test/`.

Output layer — train data.

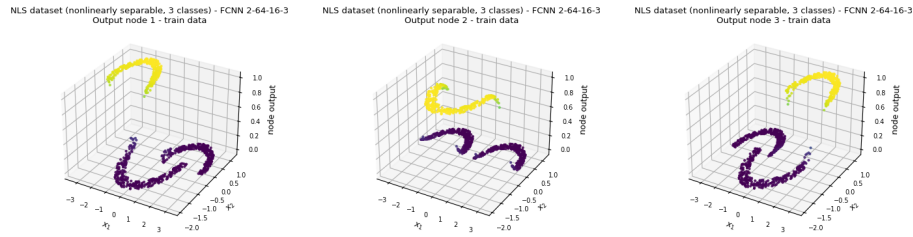


Fig. 28. Outputs of all 3 output nodes (train data), one figure per node.

Output layer — val data.

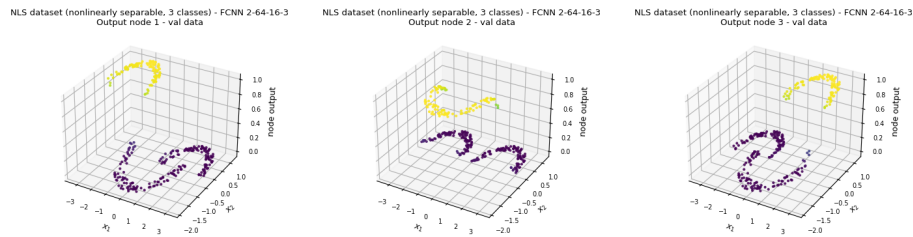


Fig. 29. Outputs of all 3 output nodes (val data), one figure per node.

Output layer — test data.

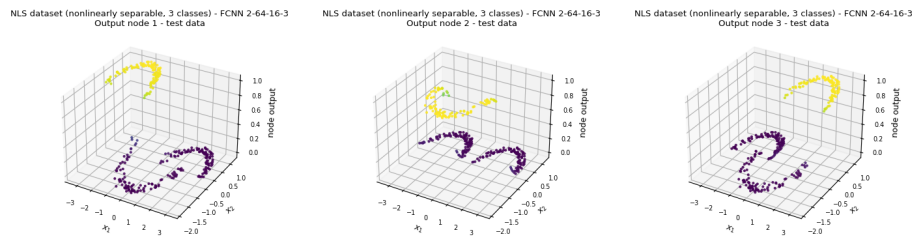


Fig. 30. Outputs of all 3 output nodes (test data), one figure per node.

3.5 Comparison with the Single-Neuron Model (Assignment-1)

The Assignment-1 model — one perceptron per pair of classes, combined by one-against-one majority voting — was retrained on exactly the same 60/20/20 split and scored with the same from-scratch metric code, so the two rows below are directly comparable.

Table 5. FCNN against the Assignment-1 single-neuron model.

Model	Val acc.%	Test acc.%	Test mean prec.	Test mean recall	Test mean F
Single neuron (one-against-one perceptron)	91.67	92.67	0.9263	0.9267	0.9264
FCNN 2-64-16-3 (Assignment-2)	100.00	100.00	1.0000	1.0000	1.0000

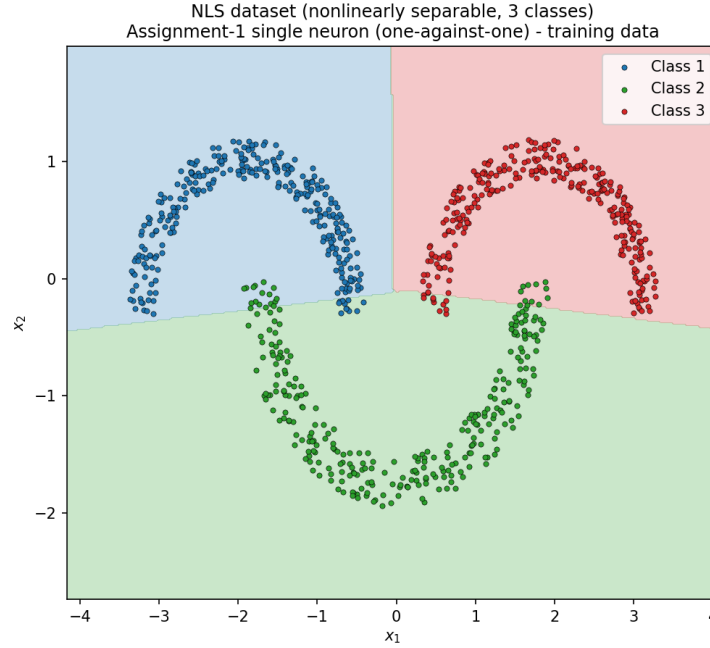


Fig. 31. Decision regions of the Assignment-1 single-neuron model on the same training data.

3.6 Inferences

- The NLS classes are interleaved arcs, so the class boundaries are strongly nonlinear. The best two-hidden-layer network, 2-64-16-3, reaches 100.00% on validation and 100.00% on test data.

- This is where the FCNN clearly beats the single neuron: the Assignment-1 one-against-one perceptron reaches only 92.67% on the same test data, i.e. the FCNN gains +7.33 percentage points. The per-class F-measure improves from 0.9264 to 1.0000.
- The reason is visible in the two decision-region plots: the single neuron can only draw straight lines, so it necessarily cuts through the arcs and misclassifies the points near the concave parts of the boundary, whereas the FCNN wraps its boundary around the arcs.
- All 4 architectures reach 100.00% validation accuracy, so two hidden layers of even modest width are already sufficient for this dataset; the extra nodes in the larger networks add parameters without adding accuracy.
- The hidden-layer output plots make the mechanism explicit: the first hidden layer produces oriented sigmoid ridges (soft half-planes), and the second hidden layer combines them into localised bump-like surfaces that follow the arcs. The output nodes are then close to 1 over one arc and close to 0 elsewhere.
- The average-error curve of the best architecture falls steeply during the first epochs and then flattens; training ran for 300 epochs before the convergence criterion stopped it. Training and validation curves stay close together, so there is no sign of overfitting at this network size.
- Training, validation and test metrics of the selected model agree closely, which is what one expects when the model is selected on data that is disjoint from the test set and the three splits are drawn from the same distribution.

4 Regression — Dataset 1 (Univariate, 1-D Input)

Model: an FCNN with one hidden layer, logistic hidden nodes and a single linear output node, trained by SGD backpropagation with the squared-error loss and learning rate $\eta = 0.01$. 4 architectures were trained and compared on the validation set.

4.1 RMSE and %RMSE for the Different Architectures

Table 6. RMSE and %RMSE on training and validation data for every architecture. The selected architecture is highlighted.

Hidden nodes	Architecture	Hidden layers	Params	Epochs	RMSE (train)	%RMSE (train)	RMSE (val)	%RMSE (val)
8	1-8-1	1	25	250	0.0590	2.314	0.0625	2.646
16	1-16-1	1	49	300	0.0635	2.489	0.0663	2.806
32	1-32-1	1	97	300	0.0667	2.613	0.0684	2.897
64	1-64-1	1	193	300	0.0685	2.684	0.0703	2.978

4.2 Best Architecture: 1-8-1

Table 7. RMSE and %RMSE of the selected architecture 1-8-1 on training, validation and test data.

Data	RMSE	%RMSE
Train	0.0590	2.3136
Val	0.0625	2.6461
Test	0.0590	2.3726

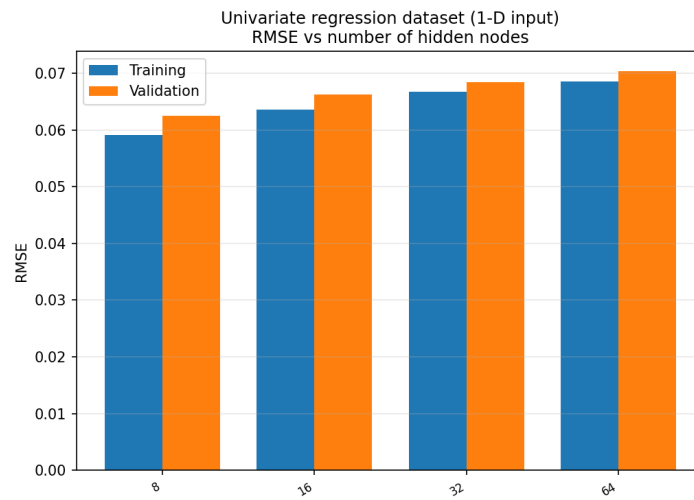


Fig. 32. RMSE against the number of hidden nodes.

Average error against epochs for each architecture that was tried, one graph per architecture. They are deliberately not overlaid on a single axis: with several architectures the curves sit on top of one another and none of them can be read.

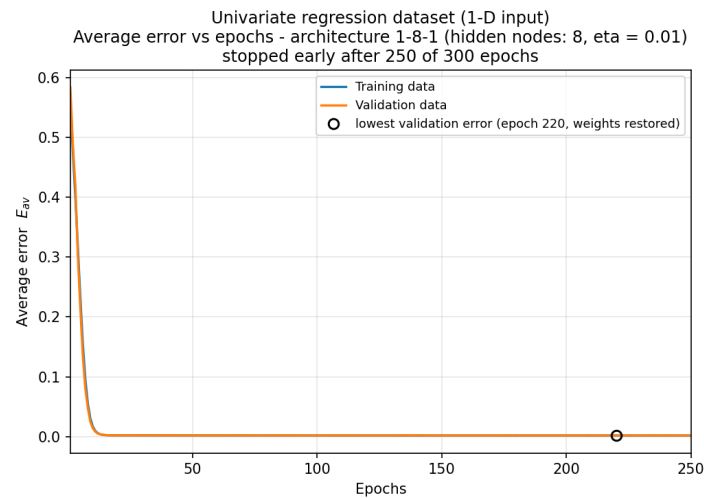


Fig. 33. Average error against epochs, architecture 1-8-1 (hidden nodes: 8).

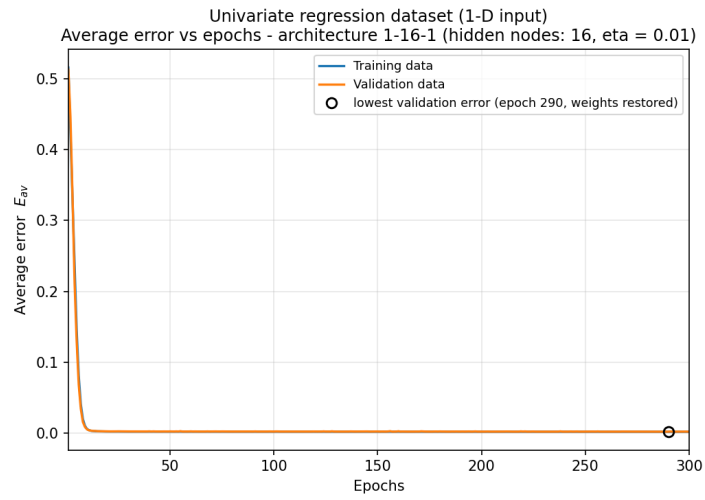


Fig. 34. Average error against epochs, architecture 1-16-1 (hidden nodes: 16).

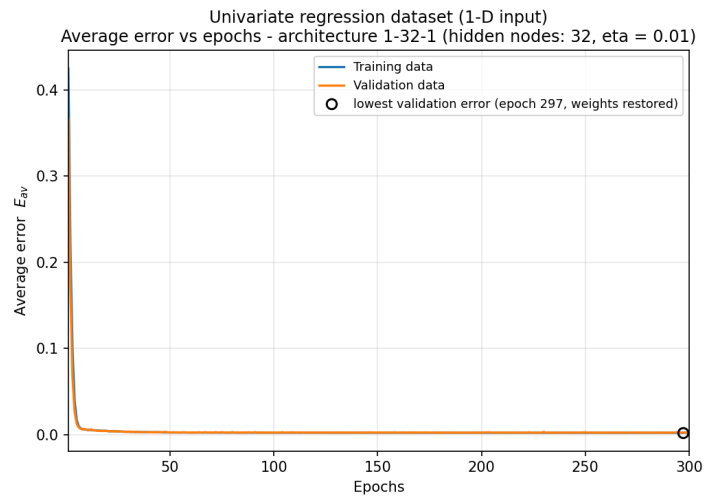


Fig. 35. Average error against epochs, architecture 1-32-1 (hidden nodes: 32).

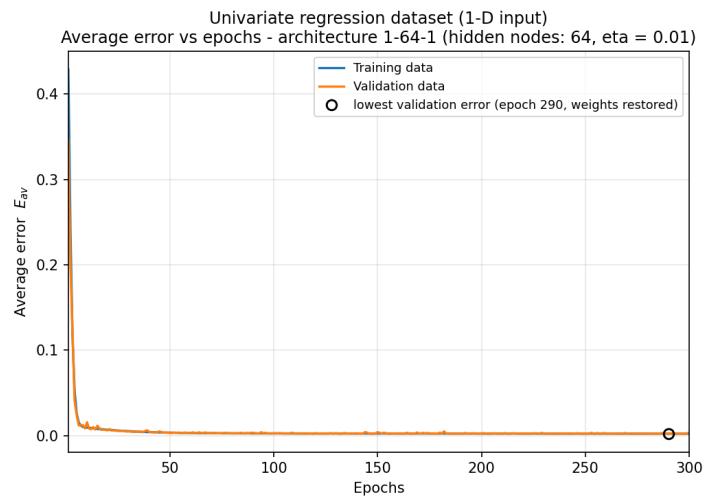


Fig. 36. Average error against epochs, architecture 1-64-1 (hidden nodes: 64).

Average error against epochs (best architecture).

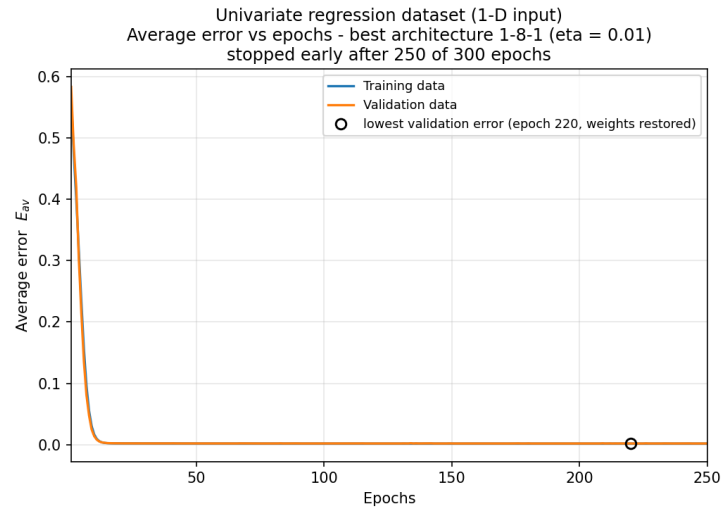


Fig. 37. Average error E_{av} against epochs for the best architecture.

4.3 Model Output Superimposed on the Target Output

In the plots below the x -axis is the input x and the y -axis is the target / model output.

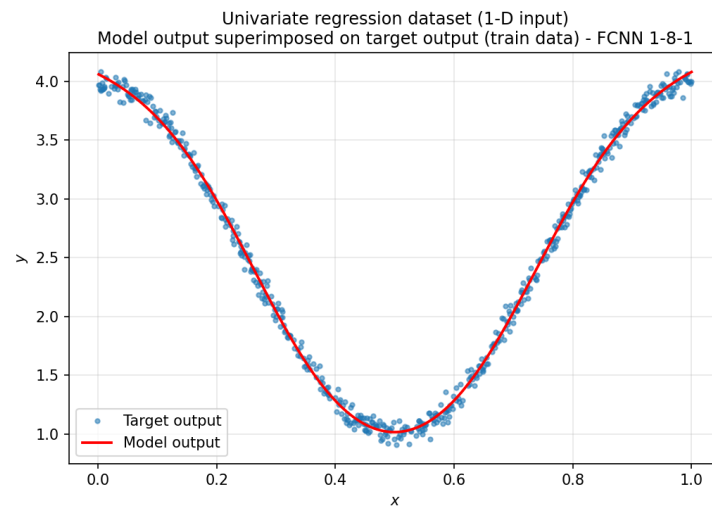


Fig. 38. Model output superimposed on the target output (train data).

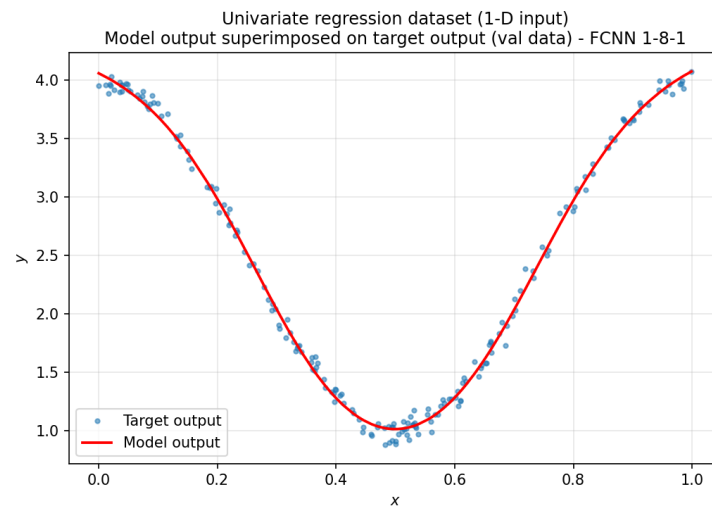


Fig. 39. Model output superimposed on the target output (val data).

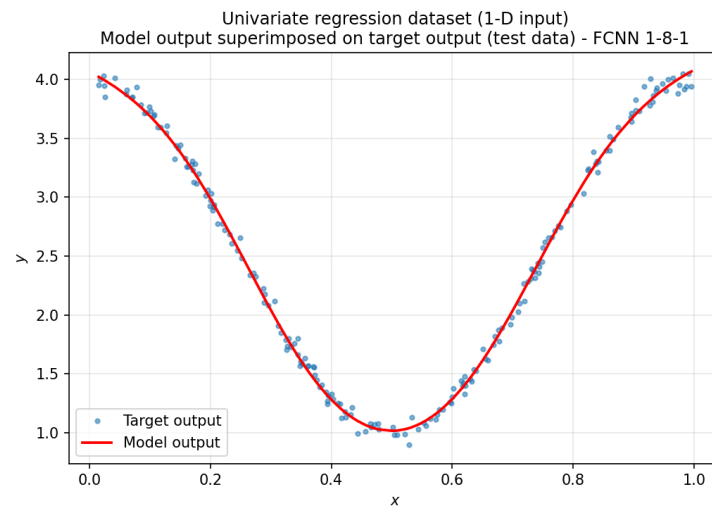


Fig. 40. Model output superimposed on the target output (test data).

4.4 Scatter Plot of Target Output against Model Output

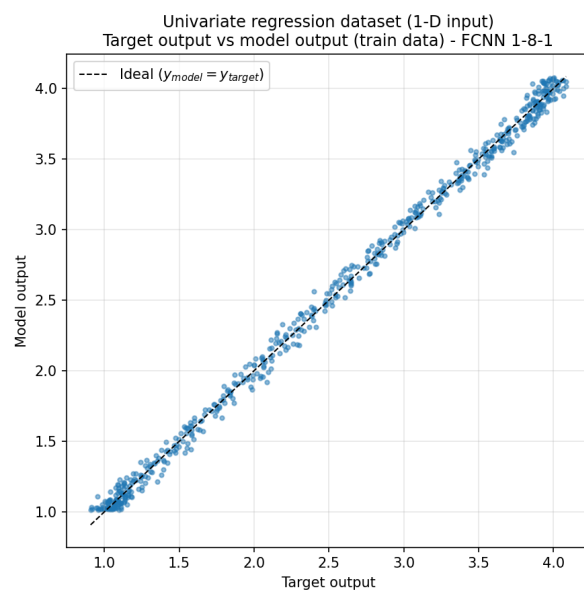


Fig. 41. Target output (horizontal axis) against model output (vertical axis), train data.

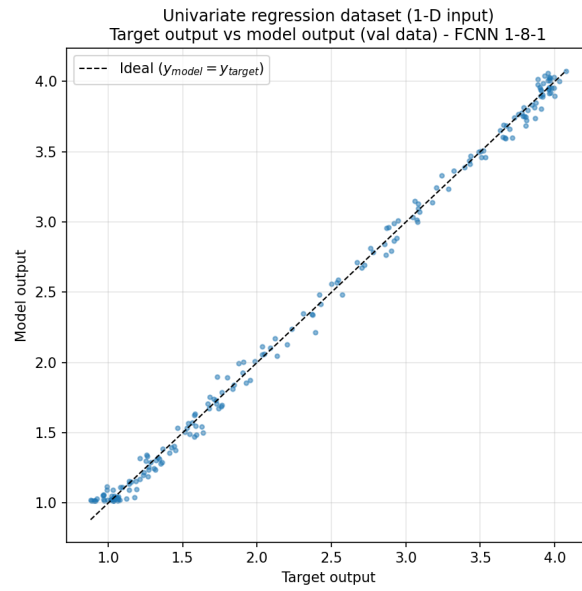


Fig. 42. Target output (horizontal axis) against model output (vertical axis), val data.

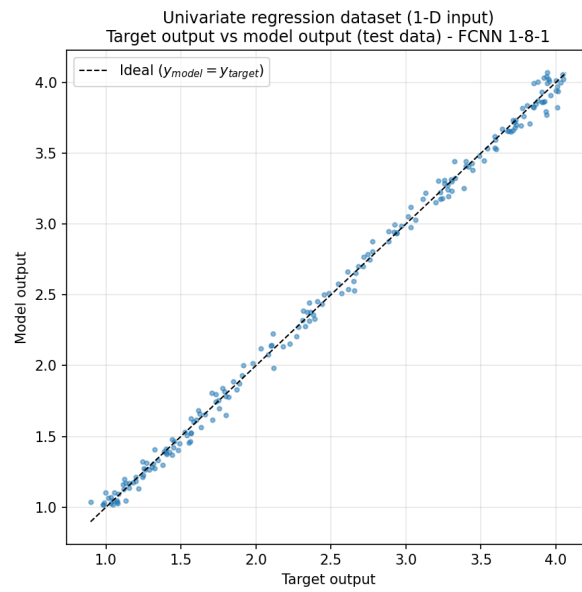


Fig. 43. Target output (horizontal axis) against model output (vertical axis), test data.

4.5 Outputs of the Hidden Nodes and Output Node

Every hidden node and the output node is plotted in its own figure: the vertical axis is the output of that node and the remaining axes are the input variables of each example. The selected network has 9 nodes in total, so the complete set is 27 figures (one per node per split); they are all written to `outputs/regression_univariate/best_architecture/node_outputs/`. The first 6 nodes of each layer are reproduced below for each split.

Hidden layer 1 — train data.

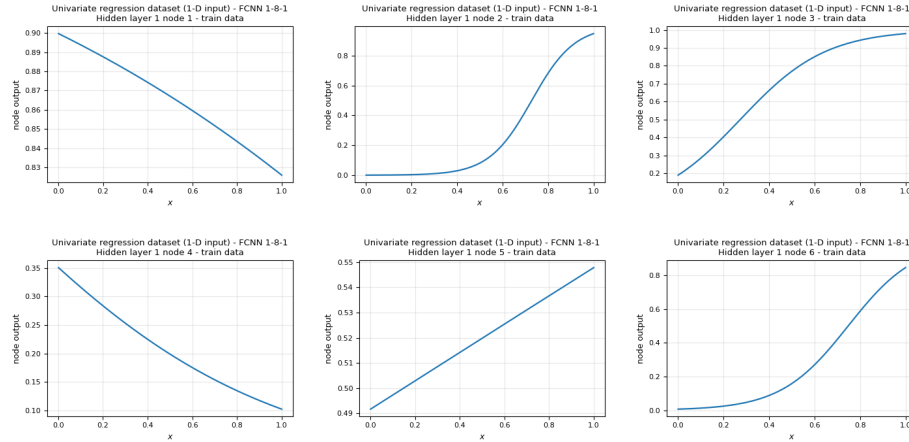


Fig. 44. Outputs of hidden layer 1 nodes 1–6 of 8 (train data). Each node has its own figure; all 8 are in `outputs/regression_univariate/best_architecture/node_outputs/hidden_layer1/train/`.

Hidden layer 1 — val data.

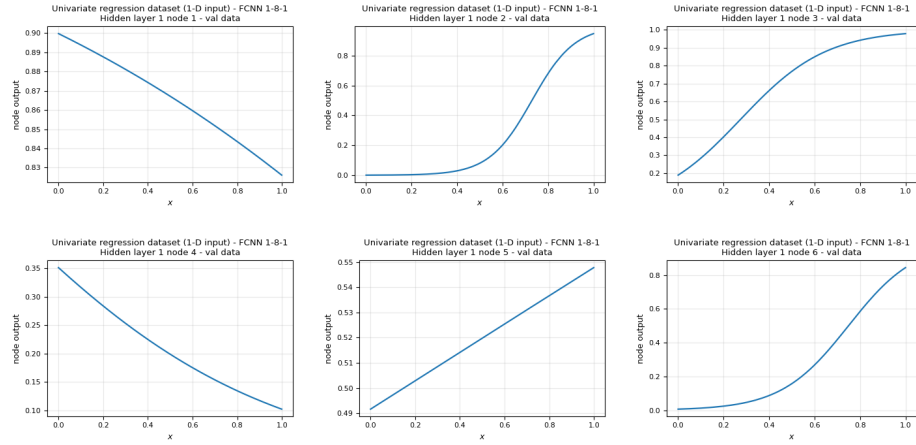


Fig. 45. Outputs of hidden layer 1 nodes 1–6 of 8 (val data). Each node has its own figure; all 8 are in `outputs/regression_univariate/best_architecture/node_outputs/hidden_layer1/val/`.

Hidden layer 1 — test data.

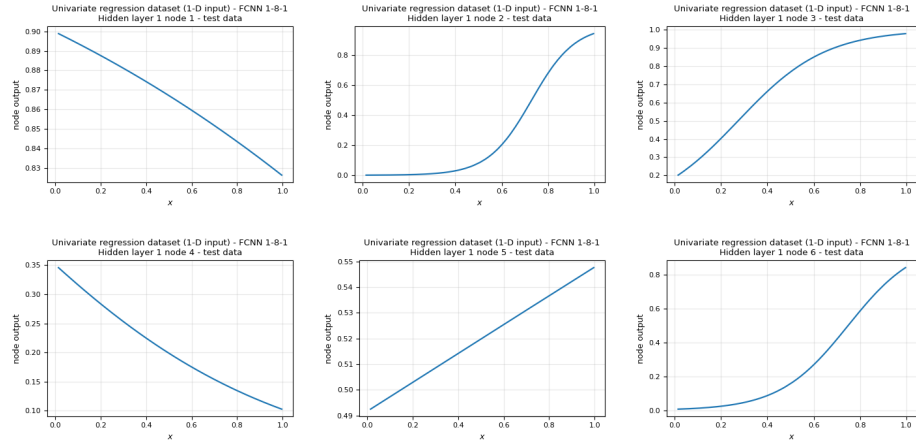


Fig. 46. Outputs of hidden layer 1 nodes 1–6 of 8 (test data). Each node has its own figure; all 8 are in `outputs/regression_univariate/best_architecture/node_outputs/hidden_layer1/test/`.

Output node — train data.

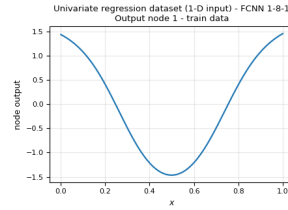


Fig. 47. Outputs of all 1 output node (train data), one figure per node.

Output node — val data.

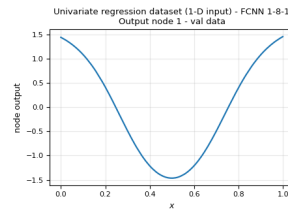


Fig. 48. Outputs of all 1 output node (val data), one figure per node.

Output node — test data.

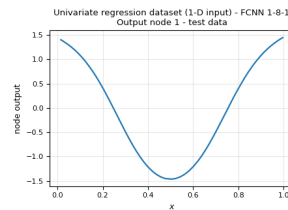


Fig. 49. Outputs of all 1 output node (test data), one figure per node.

4.6 Comparison with the Single-Neuron Model (Assignment-1)

The Assignment-1 model — a single neuron with a linear activation trained by gradient descent — was retrained on exactly the same split with the same standardisation and scored with the same from-scratch RMSE code.

Table 8. FCNN against the Assignment-1 single-neuron model.

Model	RMSE (train)	RMSE (val)	RMSE (test)	%RMSE (train)	%RMSE (val)	%RMSE (test)
Single linear neuron	1.0512	1.1349	1.0280	41.192	48.061	41.310
FCNN 1-8-1 (Assignment-2)	0.0590	0.0625	0.0590	2.314	2.646	2.373

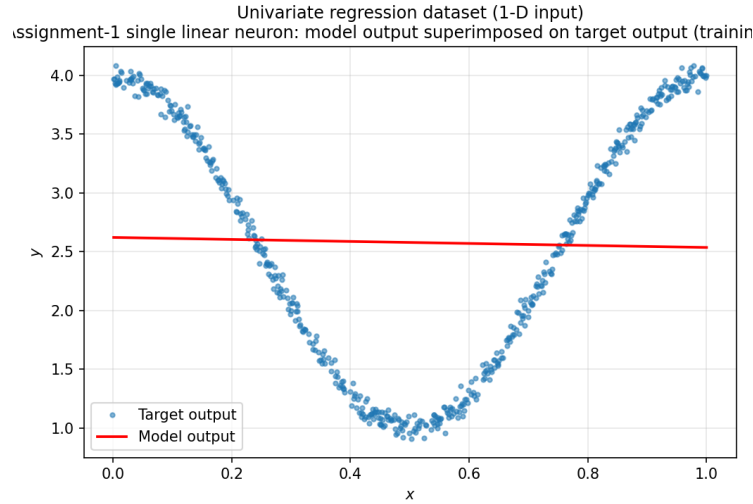


Fig. 50. The Assignment-1 single linear neuron on the same training data.

4.7 Inferences

- The selected architecture is 1-8-1, with a test RMSE of 0.0590 (2.37% RMSE). Training, validation and test RMSE (0.0590 / 0.0625 / 0.0590) are almost identical, so the selected model generalises and is not overfitted.
- The comparison with Assignment-1 is decisive: the single linear neuron reaches a test RMSE of 1.0280 (41.31%), so the FCNN reduces the test RMSE by 94.3%. A single linear neuron can only represent a hyperplane, and the target function of this dataset is not linear, so its error is dominated by bias that no amount of further training can remove.

- Increasing the number of hidden nodes helps only up to a point. The worst validation RMSE among the architectures tried is 0.0703 (architecture 1-64-1) and the best is 0.0625; beyond the selected size the extra parameters do not lower the validation error, they only make the optimisation slower and noisier. This is the model-complexity trade-off the cross-validation step is meant to expose.
- The model-output-vs-target plot shows the network reproducing the curvature of the target function over the whole input range, on training, validation and test data alike; the residual scatter around the curve is the noise in the data, not model error.
- The target-vs-model scatter plots lie tightly on the diagonal $y = x$ for all three splits, with no systematic curvature or fan-out, which confirms that the error is unbiased across the output range.
- The hidden-node output plots show that each sigmoid hidden node contributes a step / ramp positioned at a different value of x . The linear output node adds these ramps with positive and negative weights, and their sum reproduces the target curve - a direct, visual illustration of why one hidden layer of sigmoid units is a universal approximator.
- The average-error curve of the best architecture drops sharply in the first epochs and then decreases very slowly; it ran for 250 epochs. Because the weights are updated after every example, the curve is noticeably noisier than a batch-gradient-descent curve would be, which is the expected signature of SGD.

5 Regression — Dataset 2 (Bivariate, 2-D Input)

Model: an FCNN with one and with two hidden layers, logistic hidden nodes and a single linear output node, trained by SGD backpropagation with the squared-error loss and learning rate $\eta = 0.01$. 5 architectures were trained and compared on the validation set.

5.1 RMSE and %RMSE for the Different Architectures

Table 9. RMSE and %RMSE on training and validation data for every architecture. The selected architecture is highlighted.

Hidden nodes	Architecture	Hidden layers	Params	Epochs	RMSE (train)	%RMSE (train)	RMSE (val)	%RMSE (val)
16	2-16-1	1	65	200	0.2292	1.121	0.1924	0.923
32	2-32-1	1	129	200	0.3452	1.688	0.3131	1.502
64	2-64-1	1	257	200	0.3765	1.841	0.3361	1.613
32-16	2-32-16-1	2	641	200	0.1659	0.811	0.1432	0.687
64-32	2-64-32-1	2	2305	200	0.1587	0.776	0.1416	0.679

5.2 Best Architecture: 2-64-32-1

Table 10. RMSE and %RMSE of the selected architecture 2-64-32-1 on training, validation and test data.

Data	RMSE	%RMSE
Train	0.1587	0.7760
Val	0.1416	0.6794
Test	0.1483	0.7272

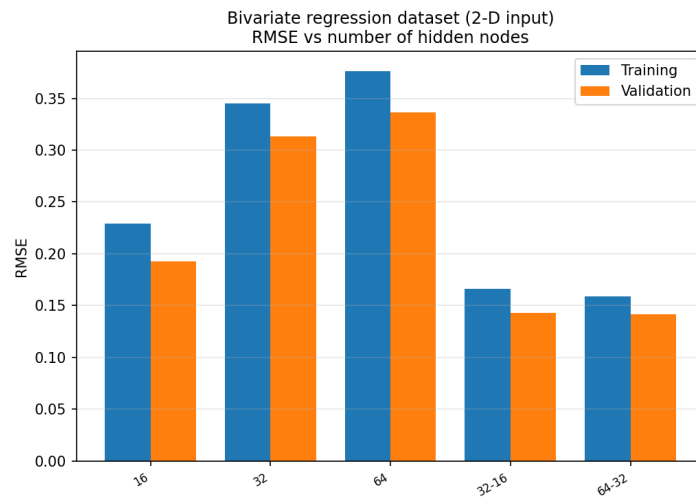


Fig. 51. RMSE against the number of hidden nodes.

Average error against epochs for each architecture that was tried, one graph per architecture. They are deliberately not overlaid on a single axis: with several architectures the curves sit on top of one another and none of them can be read.

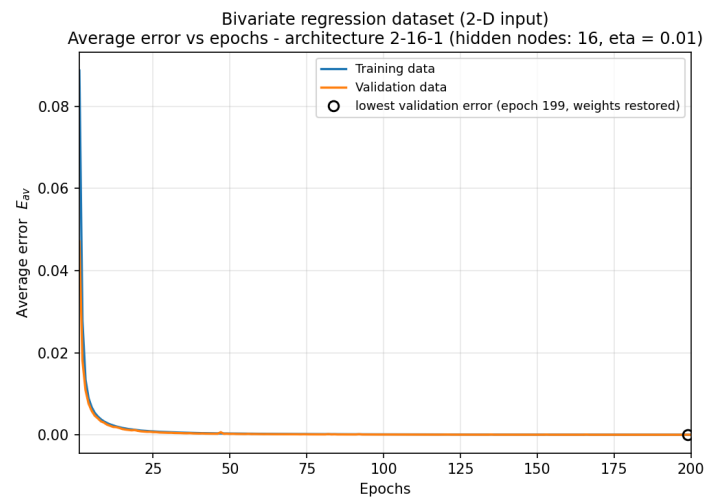


Fig. 52. Average error against epochs, architecture 2-16-1 (hidden nodes: 16).

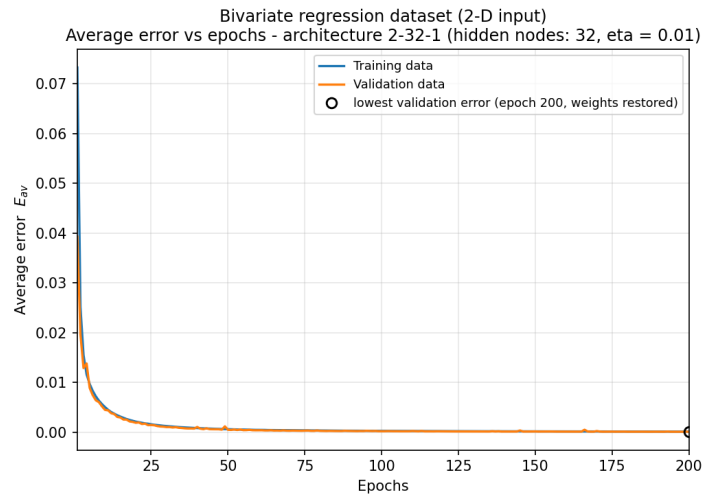


Fig. 53. Average error against epochs, architecture 2-32-1 (hidden nodes: 32).

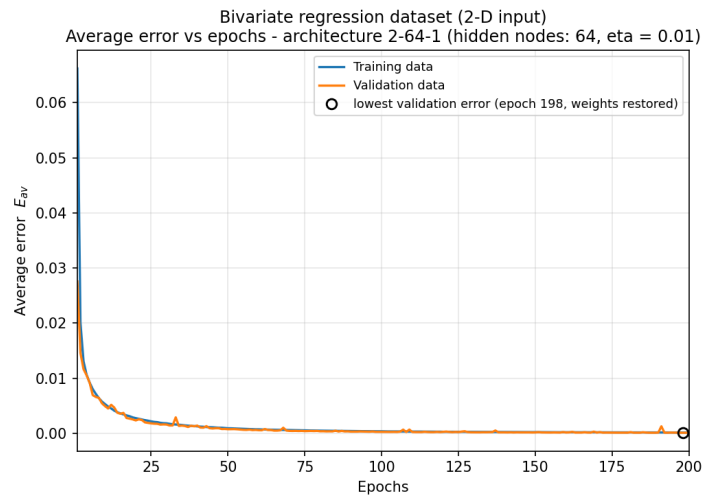


Fig. 54. Average error against epochs, architecture 2-64-1 (hidden nodes: 64).

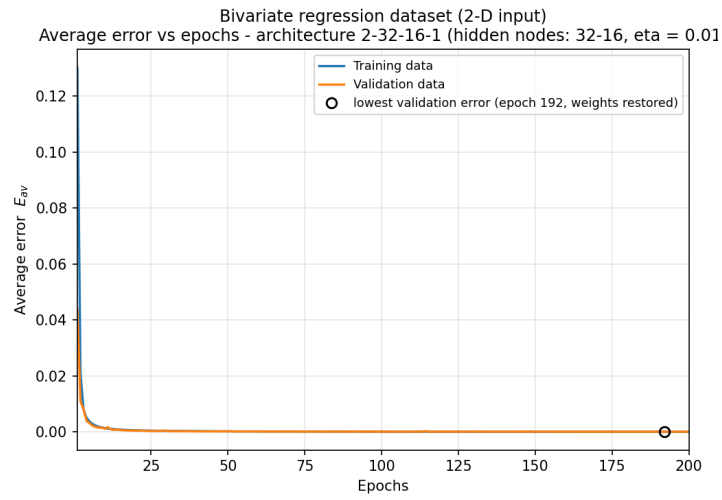


Fig. 55. Average error against epochs, architecture 2-32-16-1 (hidden nodes: 32-16).

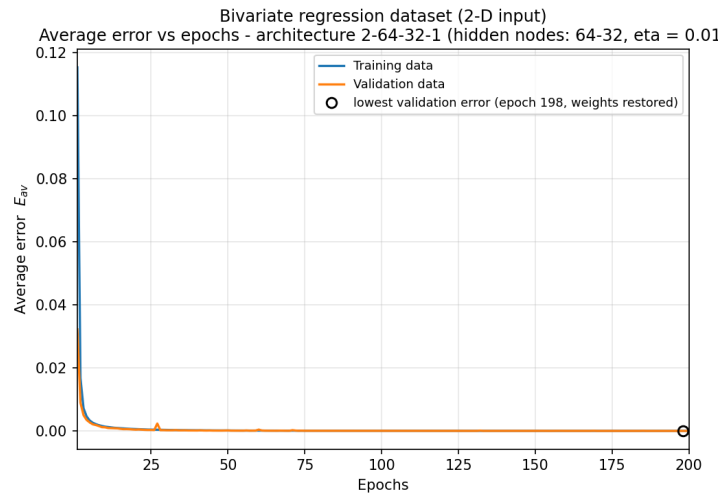


Fig. 56. Average error against epochs, architecture 2-64-32-1 (hidden nodes: 64-32).

Average error against epochs (best architecture).

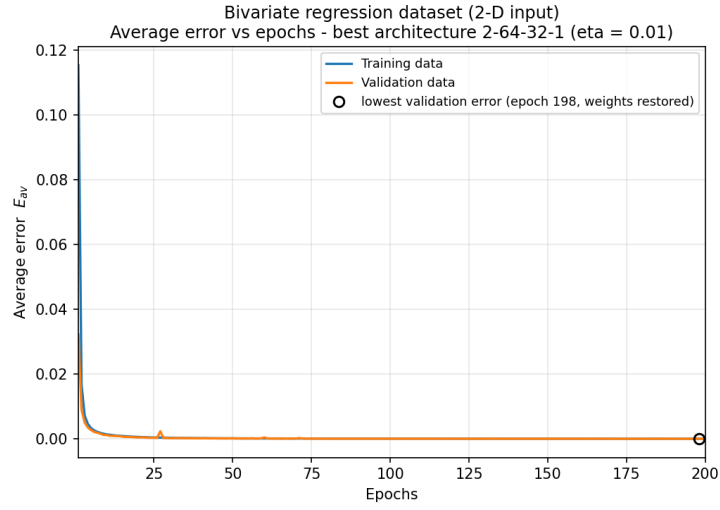


Fig. 57. Average error E_{av} against epochs for the best architecture.

5.3 Model Output Superimposed on the Target Output

In the plots below the two base axes are x_1 and x_2 and the vertical axis is the target / model output.

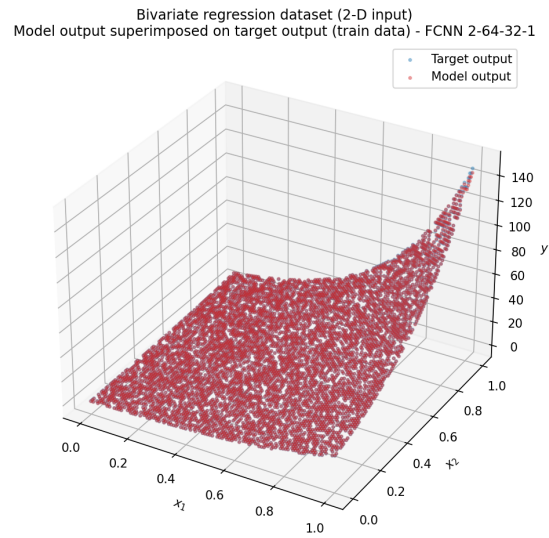


Fig. 58. Model output superimposed on the target output (train data).

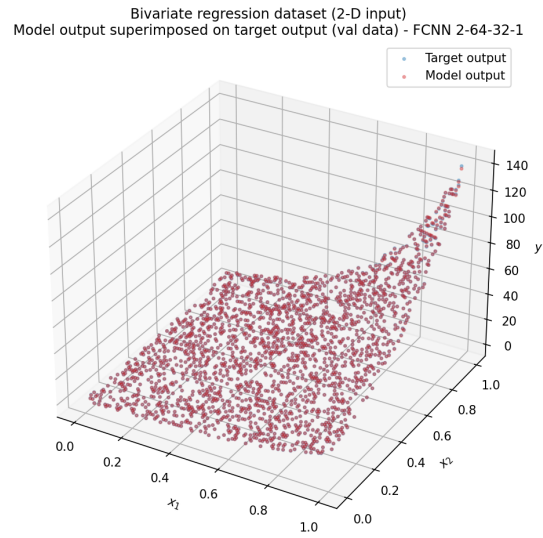


Fig. 59. Model output superimposed on the target output (val data).

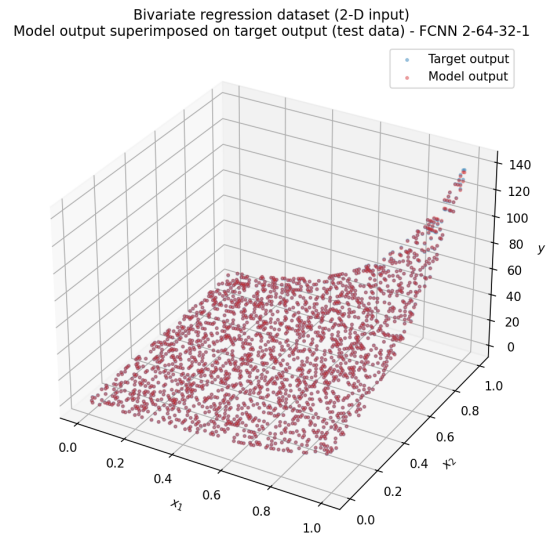


Fig. 60. Model output superimposed on the target output (test data).

5.4 Scatter Plot of Target Output against Model Output

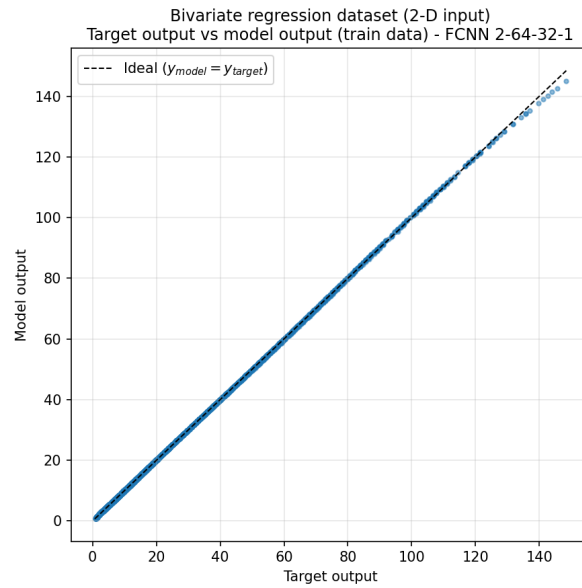


Fig. 61. Target output (horizontal axis) against model output (vertical axis), train data.

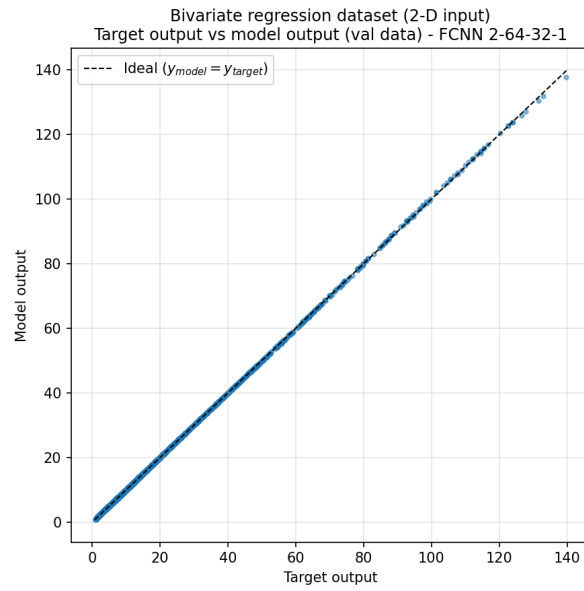


Fig. 62. Target output (horizontal axis) against model output (vertical axis), val data.

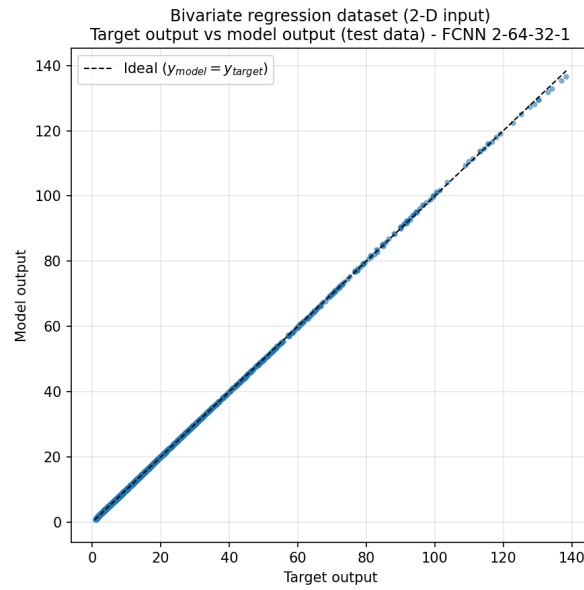


Fig. 63. Target output (horizontal axis) against model output (vertical axis), test data.

5.5 Outputs of the Hidden Nodes and Output Node

Every hidden node and the output node is plotted in its own figure: the vertical axis is the output of that node and the remaining axes are the input variables of each example. The selected network has 97 nodes in total, so the complete set is 291 figures (one per node per split); they are all written to `outputs/regression_bivariate/best_architecture/node_outputs/`. The first 6 nodes of each layer are reproduced below for each split.

Hidden layer 1 — train data.

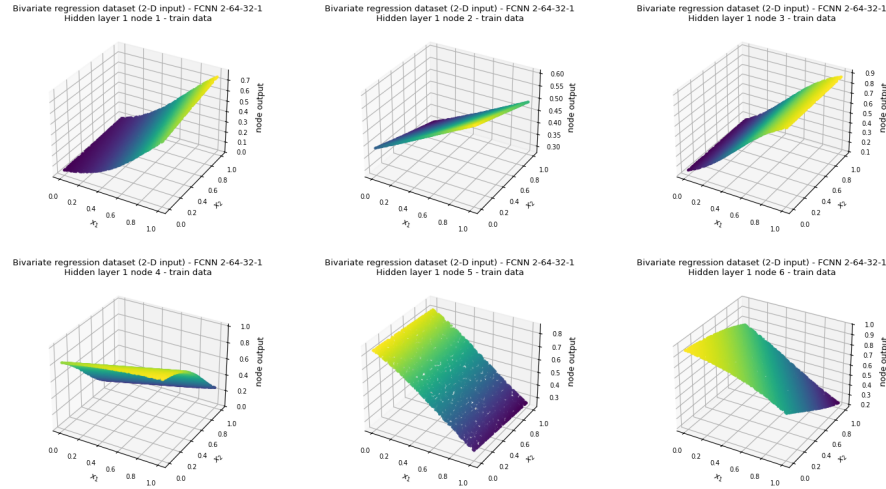


Fig. 64. Outputs of hidden layer 1 nodes 1–6 of 64 (train data). Each node has its own figure; all 64 are in `outputs/regression_bivariate/best_architecture/node_outputs/hidden_layer1/train/`.

Hidden layer 1 — val data.

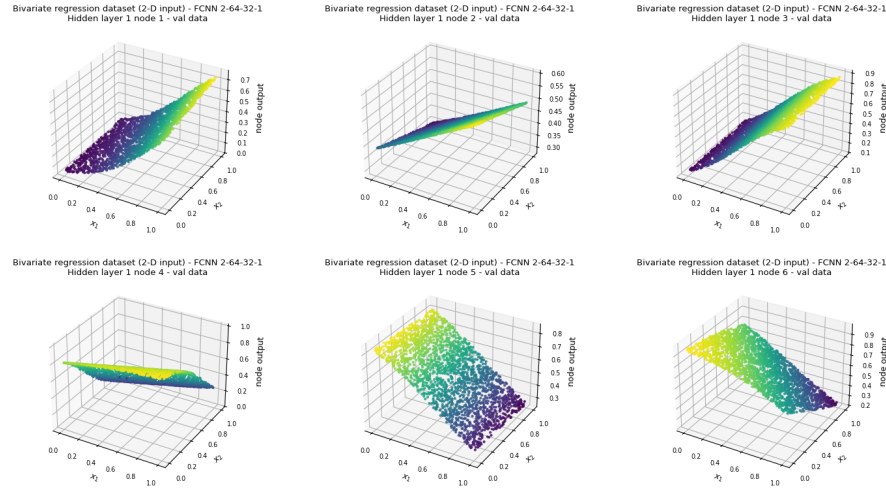


Fig. 65. Outputs of hidden layer 1 nodes 1–6 of 64 (val data). Each node has its own figure; all 64 are in `outputs/regression_bivariate/best_architecture/node_outputs/hidden_layer1/val/`.

Hidden layer 1 — test data.

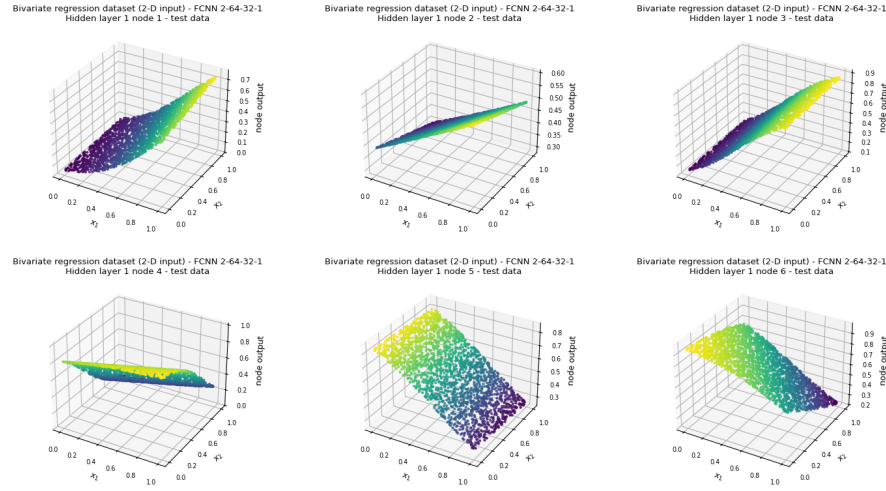


Fig. 66. Outputs of hidden layer 1 nodes 1–6 of 64 (test data). Each node has its own figure; all 64 are in `outputs/regression_bivariate/best_architecture/node_outputs/hidden_layer1/test/`.

Hidden layer 2 — train data.

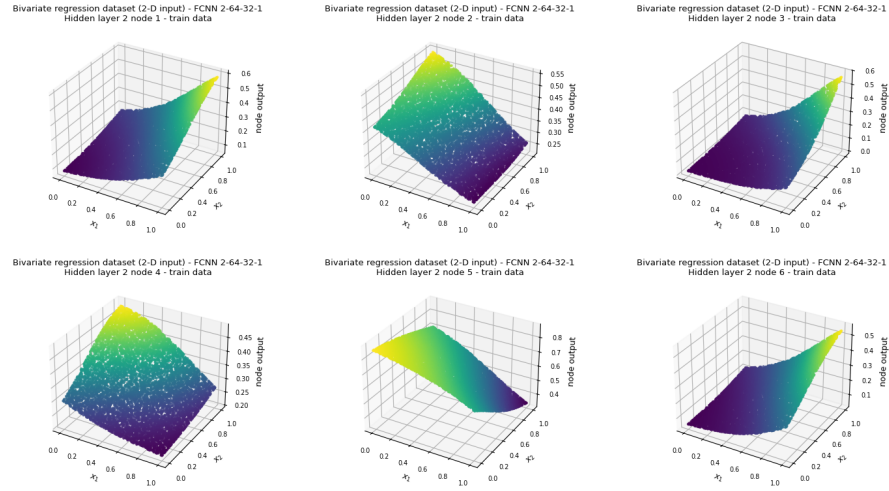


Fig.67. Outputs of hidden layer 2 nodes 1–6 of 32 (train data). Each node has its own figure; all 32 are in `outputs/regression_bivariate/best_architecture/node_outputs/hidden_layer2/train/`.

Hidden layer 2 — val data.

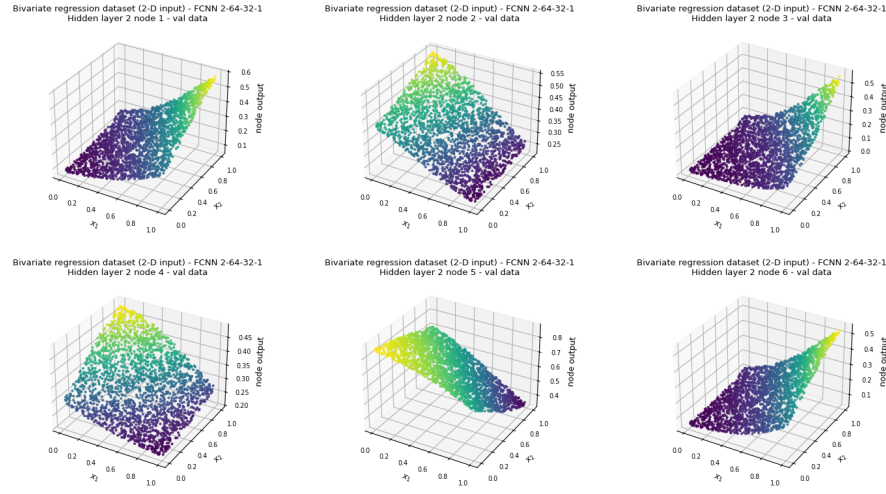


Fig. 68. Outputs of hidden layer 2 nodes 1–6 of 32 (val data). Each node has its own figure; all 32 are in `outputs/regression_bivariate/best_architecture/node_outputs/hidden_layer2/val/`.

Hidden layer 2 — test data.

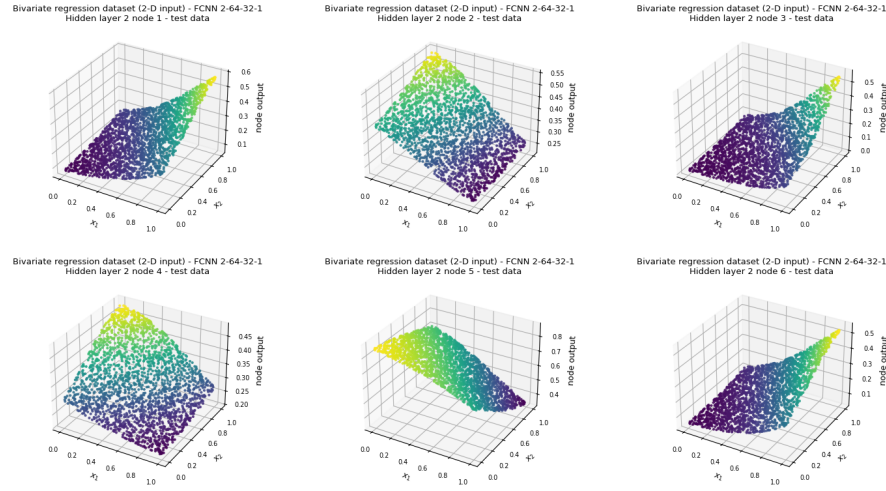


Fig. 69. Outputs of hidden layer 2 nodes 1–6 of 32 (test data). Each node has its own figure; all 32 are in `outputs/regression_bivariate/best_architecture/node_outputs/hidden_layer2/test/`.

Output node — train data.

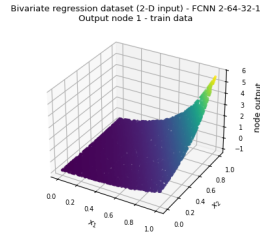


Fig. 70. Outputs of all 1 output node (train data), one figure per node.

Output node — val data.

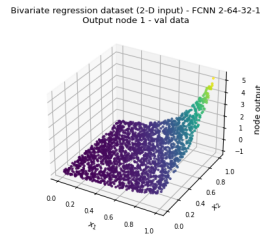


Fig. 71. Outputs of all 1 output node (val data), one figure per node.

Output node — test data.

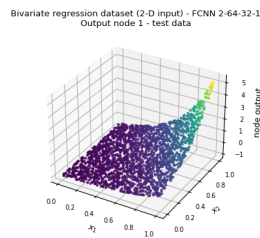


Fig. 72. Outputs of all 1 output node (test data), one figure per node.

5.6 Comparison with the Single-Neuron Model (Assignment-1)

The Assignment-1 model — a single neuron with a linear activation trained by gradient descent — was retrained on exactly the same split with the same standardisation and scored with the same from-scratch RMSE code.

Table 11. FCNN against the Assignment-1 single-neuron model.

Model	RMSE (train)	RMSE (val)	RMSE (test)	%RMSE (train)	%RMSE (val)	%RMSE (test)
Single linear neuron	11.5134	11.7742	11.5259	56.297	56.481	56.508
FCNN 2-64-32-1 (Assignment-2)	0.1587	0.1416	0.1483	0.776	0.679	0.727

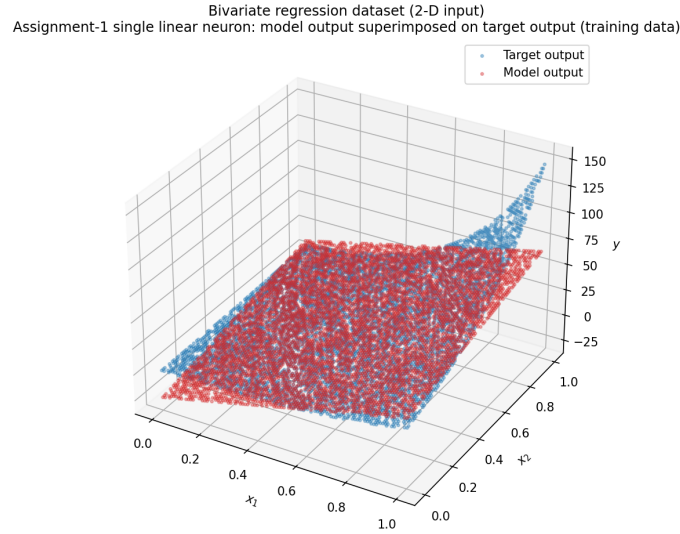


Fig. 73. The Assignment-1 single linear neuron on the same training data.

5.7 Inferences

- The selected architecture is 2-64-32-1, with a test RMSE of 0.1483 (0.73% RMSE). Training, validation and test RMSE (0.1587 / 0.1416 / 0.1483) are almost identical, so the selected model generalises and is not overfitted.
- The comparison with Assignment-1 is decisive: the single linear neuron reaches a test RMSE of 11.5259 (56.51%), so the FCNN reduces the test RMSE by 98.7%. A single linear neuron can only represent a hyperplane, and the target function of this dataset is not linear, so its error is dominated by bias that no amount of further training can remove.

- Increasing the number of hidden nodes helps only up to a point. The worst validation RMSE among the architectures tried is 0.3361 (architecture 2-64-1) and the best is 0.1416; beyond the selected size the extra parameters do not lower the validation error, they only make the optimisation slower and noisier. This is the model-complexity trade-off the cross-validation step is meant to expose.
- The 3-D plot of model output superimposed on the target output shows the network surface following the target surface over the whole (x_1, x_2) domain, whereas the single linear neuron can only fit a flat plane through the same data.
- The target-vs-model scatter plots stay close to the diagonal for training, validation and test data; the small remaining spread is uniform across the output range rather than concentrated at the extremes.
- The hidden-node output plots show oriented sigmoid ramps over the (x_1, x_2) plane. With two hidden layers the second layer produces more localised surfaces, which is what lets the network model the curvature of the target surface with relatively few nodes.
- The assignment asks for both one and two hidden layers on this dataset. The best one-hidden-layer network is 2-16-1 (validation RMSE 0.1924, 65 weights) and the best two-hidden-layer network is 2-64-32-1 (validation RMSE 0.1416, 2305 weights), so the two-hidden-layer family wins here by 0.0508 RMSE - 2-64-32-1 against 2-16-1. The second hidden layer lets the network build more localised surfaces from the ridges produced by the first layer, which suits the curvature of this target function; note that it wins with a comparable, not a larger, parameter count, so the gain comes from the arrangement of the nodes rather than from sheer size.
- The average-error curve of the best architecture drops sharply in the first epochs and then decreases very slowly; it ran for 200 epochs. Because the weights are updated after every example, the curve is noticeably noisier than a batch-gradient-descent curve would be, which is the expected signature of SGD.

6 Overall Observations and Conclusions

- **Nonlinear problems are where the FCNN pays off.** On the linearly separable classification dataset the FCNN and the Assignment-1 single neuron both reach 100.00% test accuracy, so the extra capacity is unused. On the nonlinearly separable dataset the FCNN reaches 100.00% against 92.67% for the single neuron, and on the two regression datasets it reduces the test RMSE by 94.3% and 98.7% respectively.
- **Hidden nodes are feature detectors.** The hidden-node output plots show soft half-planes (classification) and ramps (regression); the output layer is a linear combination of those surfaces passed through the output activation. The number of hidden nodes therefore controls how many such surfaces are available, i.e. the capacity of the model.
- **More nodes is not better.** Validation accuracy / RMSE stops improving - and occasionally worsens - once the network is large enough. Selecting the architecture on validation data, and only then reporting on the test set, is what keeps the reported figures honest.
- **SGD behaves as expected.** The average-error curves fall steeply for the first few epochs and then flatten, and they are visibly noisy because the weights are updated after every single example. Standardising the inputs was necessary: with the raw inputs the logistic units saturate and the curves barely move.
- **Sensitivity to initialisation and to the optimisation run.** Performance is not monotone in the number of parameters. On the bivariate dataset, for example, 2-64-1 (257 weights) ends at a validation RMSE of 0.3361 while the smaller 2-16-1 (65 weights) reaches 0.1924 - the larger network can represent everything the smaller one can, so the gap is a property of the training run, not of the model. Squared error with logistic units has long flat regions, and plain SGD from a random start does not always escape them. This is a real limitation of the from-scratch setup and is why several architectures were trained and compared rather than one.